# Final Report

## *ECOMATCH AI* – Automated Product Inquiry Response System for Light Recycling Program

**Student Name**: Namesh Mathara Arachchi Vidanalage

**Student ID**: 300359798

**Course**: CSIS 4495 – Applied Research Project

**Section**: 002

# Contents

# Checklist

| | |
|---|---|
| ☒ | Final Implementation **GitHub Repository Code Check-In** |
| ☒ | Presentation Slides **Completed and Checked into GitHub under ReportsAndDocuments** |
| ☒ | **Final Defense and Demo Preparation** |
| ☒ | Installation Instructions/Guide |
| ☒ | User Instructions/Guide |
| ☒ | **Final Report Blackboard Submission** |
| ☒ | **GitHub Repository Final Report Check-in** |

# Introduction

## Background and Context

Product Care Association (PCA) is a leading industry-led organization dedicated to environmental protection through recycling programs for post-consumer products such as paint, household hazardous waste, lights, and alarms. This research focuses specifically on light recycling and aims to develop an automated Product Inquiry Response System to enhance operational efficiency.

PCA successfully diverted over 10.8 million light bulbs from landfills in 2022, operating across multiple Canadian provinces, including British Columbia, Manitoba, Ontario, Quebec, Nova Scotia, and Prince Edward Island. The organization collaborates with recycling centers, retail chains, and bottle depots to facilitate the recycling of lighting products. However, the current product inquiry process remains largely manual, leading to inefficiencies in response times, inconsistencies in decision-making, and increased operational workload.

## Problem Statement and Research Questions

Before accepting a product for recycling, PCA members must determine whether it falls under the approved recycling program and if it does, the relevant product category. This verification process involves consulting a Product Guide or manually searching through historical product inquiries stored in Excel sheets. The existing approach presents several inefficiencies:

- Repetitive inquiries from different members for the same product.
- Manual searches through extensive product records are time-consuming.
- Inconsistent decision-making, as different staff members handle inquiries differently.

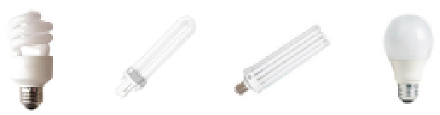
*A screenshot of past product inquiries stored in Excel sheet:*

| | A | B | C | D | E | F | G | H |
|---|---|---|---|---|---|---|---|---|
| 1 | Date Received | Photo | Product Name | Product Description (from web if possible) | Link | Product Type | BC | Rationale |
| 12 | 2019-09-11 |  | Globe Electric 01201 55-watt Circline Soft White, CFL 4 Pin Base Replacement Light Bulb, 150-watt Equivalent | Circline cfl 4-pin base replacement light bulb<br>Uses only 55 watts of energy with an equivalent output of 150 watts<br>Light output of 3,300 lumens<br>Lifespan of 10,000 hours<br>Used for replacement lighting | https://www.amazon.ca/Globe-Electric-01201-Replacement-Equivalent/dp/B00E383WLY | CFL lamp | Included (category 4) | The use is as a bulb/lamp due to the screw in base |
| 13 | 2019-09-11 |  | circular fluorescent tube with screw base | circular fluorescent tube with screw base | | CFL lamp | Included (category 4) | The use is as a bulb/lamp due to the screw in base |
| 14 | 2019-09-05 |  | Nanoleaf Rhythm Smarter LED Light Panel Kit | Rhythm Upgrade Module senses audio and transforms it into light for a stunning real-time audiovisual display<br>Includes16.7-million colours as well as tunable white (candlelight to daylight) dimmable light | https://www.amazon.ca/Nanoleaf-Rhythm-Smarter-Light-NL28-2003TW-9PK/dp/B079FZGZS4 | Decorative LED Panel Lights | Excluded | Primary purpose is decorative, not to illuminate space |
| 15 | 2019-08-02 |  | Bicycle Spoke Lights | Ensure the rider is seen at night. Used for safety and decoration | | Bike Spoke Lights | Excluded | Primary purpose is not to illuminate space. It's purpose is to ensure the rider is seen, they light up the bike not the space |

*An extract from the Product Catalogue:*

**Light Recycling Product Guide**

## British Columbia

| CATEGORY | DESCRIPTION | EHF |
|---|---|---|
| 1. Fluorescent tubes measuring less than or equal to 2 ft | Includes all diameters and light outputs, shaped fluorescent tubes, and UV-A and UV-B tubes.<br> | |
| 2. Fluorescent tubes measuring greater than 2 ft and up to or equal to 4 ft | | |
| 3. Fluorescent tubes measuring greater than 4 ft | | |
| 4. Compact Fluorescent Lights (CFL)/ Screw-In Induction Lamps | Fluorescent bulbs that are typically similar in size and intended to replace an incandescent (traditional) light bulb, including pin-type sockets, covered CFLs and various output wattages. Includes screw-in induction lamps.<br> | |
| 5. Light Emitting Diodes | Solid-state lamps used for specialty purposes and conventional lighting applications.<br> | |

British Columbia

Manitoba

Quebec

Prince Edward Island

This research seeks to automate and optimize the product inquiry process by answering the following key question:

> ***"How can I design and implement an end-to-end AI-powered system that effectively integrates text and image data to enhance matching accuracy and minimize response times in the current product inquiry process?"***

## Existing Knowledge Gaps and Research Challenges

While PCA maintains a Product Guide as the primary reference, its complex structure, inconsistent formatting, and lack of a standardized database present challenges for automation. My research into text and image extraction techniques has revealed that:

- The Product Guide does not follow a uniform tabular format, making direct extraction difficult.
- Merged cells, inconsistent labeling, and multiple images within single rows complicate automated data processing.
- Manual verification remains necessary to ensure data integrity, as fully automating extraction would require extensive custom handling.

Additionally, historical product inquiries stored in Excel sheets suffer from data integrity issues, including inconsistent category labeling, formatting errors, and missing values. For this reason, I have finished the project by using the product guide to create the products table. Once the issues in the spreadsheet have been addressed, I can merge its data into the products table.

## Relevant Literature and Research Foundations

This study builds upon existing research in Natural Language Processing (NLP), Computer Vision (CV), and database optimization for structured data retrieval. Key areas of investigation include:

- Text Processing: Utilizing BERT-based sentence embeddings for contextual product name/description matching.
- Image Processing: Employing CNN models (ResNet50/EfficientNet) to compare product images for similarity-based retrieval.
- Hybrid Decision Models: Combining text and image similarity scores to improve accuracy in classifying product inquiries.

Research has shown that NLP models like BERT excel at contextual understanding, making them ideal for product description comparisons. Likewise, CNN-based image recognition models can accurately classify product images, making them suitable for matching product inquiries based on visual similarities. This research aims to bridge the gap by integrating both approaches into a single AI-driven system.

## Hypotheses, Assumptions, and Potential Benefits

### *Hypotheses and Assumptions*

- A hybrid NLP + CV ML model will accurately match product inquiries to PCA's product database.
- Automating inquiries will reduce response times by at least 50%.
- Sufficient historical data exists to train an effective ML model.
- Product images provided in inquiries will be of reasonable quality for analysis.

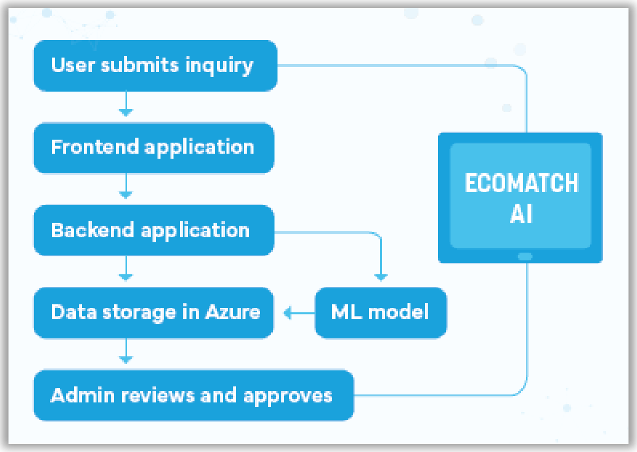
### *Potential Benefits*

This research will yield significant benefits for PCA and its recycling partners, including:

- Operational Efficiency: Automating the inquiry process will reduce manual workload, allowing staff to focus on higher-value tasks.
- Faster Response Times: Members will receive instantaneous responses, improving service levels.
- Consistency in Decision-Making: AI-driven classification will minimize human error and ensure uniform responses.
- Scalability: The system can be expanded to support other recyclable product categories beyond lighting.
- Cost Savings: Automating the process will reduce operational expenses linked to manual handling.
- AI-Driven Innovation: This project represents PCA's first ML-powered operational system, paving the way for future AI implementations.

# Project Summary

This project successfully developed an AI-powered product inquiry system that automates the identification of recyclable products using machine learning and a web-based interface. It was carried out across seven key phases, ensuring a structured and efficient implementation.




## Project Phases

1. Defining the Project and Scope – Establishing project objectives, requirements, and key deliverables.
2. Product Guide Data Extraction – Extracting structured data from the product guide (PDF).
3. Web Portal Development – Building a Flask-based portal for product inquiries.
4. ML Model - Text Matching – Implementing BERT-based text processing for product name/description matching.
5. ML Model - Image Matching – Utilizing CNN models for image-based product identification.
6. System Integration & Testing – Combining all components and ensuring interoperability.
7. Deployment, Security & Final Testing – Securing, optimizing, and deploying the solution.

## Tech Stack

- Frontend: Developed using **Python Flask** and **Bootstrap** to provide a user-friendly and responsive interface.
- Backend: **Python Flask** handles API requests and integrates machine learning models, while **Azure Tables** store inquiry data.
- Data Storage: Images are stored in **Azure Blob Storage**, while textual data are managed in **Azure Table Storage**.
- Text Matching: BERT-based models - ***all-mpnet-base-v2*** and ***bert-large-nli-stsb-mean-tokens*** generate contextual embeddings for product names and descriptions, ensuring accurate text similarity detection.
- Image Matching: CNN model - ***ResNet50*** extracts image features and compare them with stored product images.
- Decision Model: An ensemble approach combines text and image similarity scores, improving classification accuracy and robustness.

## Used Paid Services

- Azure Subscription – Azure Table Storage & Azure Blob Storage

# Changes in the Project Implementation and Justifications

## Reordering Project Phases

- **Change**: Initially, the plan was to develop the ML model first and then build the web portal. However, the web portal has been developed first, and ML implementation will follow.
- **Justification**: Building the web portal first provided a clearer end-to-end perspective of the project. This approach also allows for early feedback from employees, ensuring that the portal aligns with business needs before integrating the ML model.

## Modifications in Product Guide Data Extraction

- **Change**: Initially planned to use both the Product Guide (PDF) and past product inquiry data (Excel). Currently, only the Product Guide is being used due to data integrity issues in the spreadsheet.
- **Justification**: The Excel data contains inconsistencies that need to be addressed to maintain dataset quality for ML training. Since data integrity significantly impacts ML model performance, the focus has been on structuring the Product Guide first. The spreadsheet data will be integrated later once cleaned.

## Adjustments in PDF Data Extraction Approach

- **Change**: Initially proposed automated extraction using PyMuPDF and Tesseract OCR. Now, data is being manually extracted and formatted in Excel using Adobe Acrobat Pro.
- **Justification**: The Product Guide has a complex table structure with merged cells, inconsistent labeling, and multiple images per row, making automated extraction unreliable. Given that the dataset has only 238 records, manual formatting is more efficient than developing custom scripts to handle inconsistencies. This ensures higher accuracy with less effort.

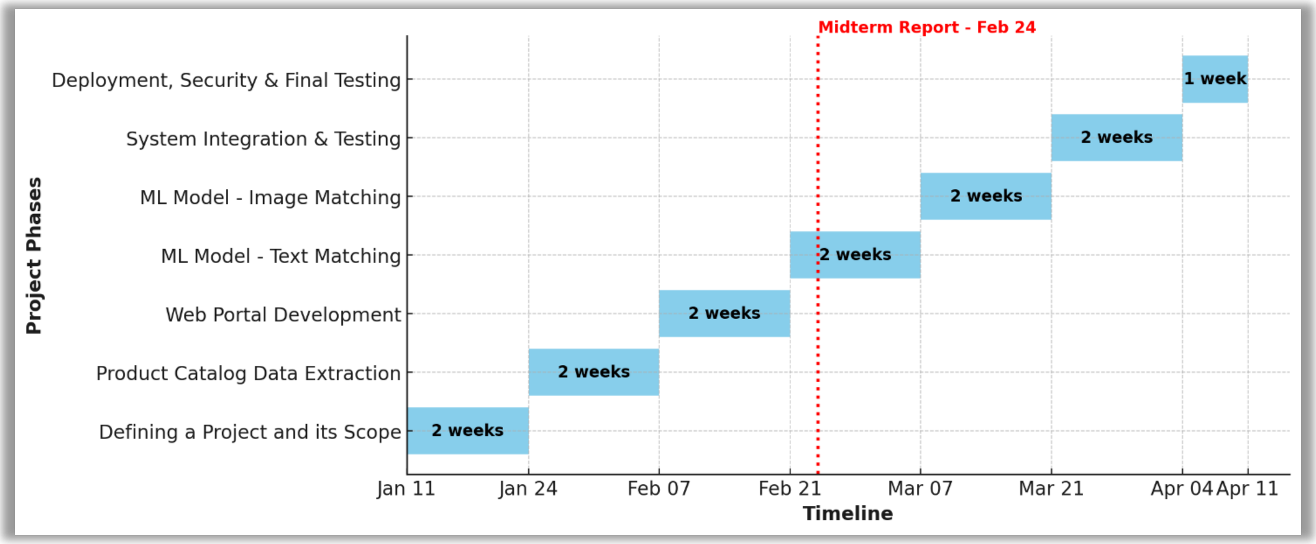
## Change in Backend Data Storage Selection

- **Change**: Originally planned to store inquiry data in Azure SQL Server. Now, Azure Table Storage is being used instead.
- **Justification**: The data structure is simple, and the inquiry frequency is low, making Azure SQL Server unnecessary. Azure Table Storage is significantly more cost-effective, scalable, and sufficient for the current use case.

# Project Planning and Timeline

As mentioned in the previous section, the only modification is the decision to develop the Web Portal before the ML model. Other than this adjustment, the timeline remained unchanged, including the time allocated for each phase. See below final timeline and Gantt chart.

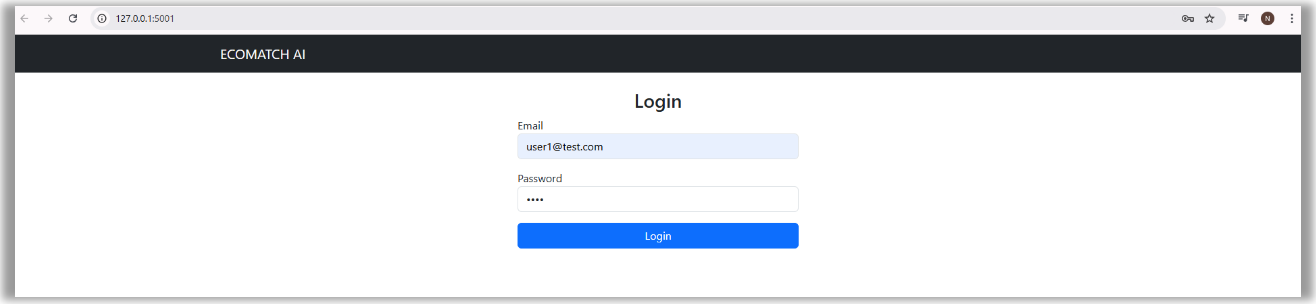
| Phase | Start Date | End Date | Duration (Weeks) |
|---|---|---|---|
| **Defining a Project and its Scope** | 2025-01-11 | 2025-01-24 | 2 |
| **Product Catalog Data Extraction** | 2025-01-24 | 2025-02-07 | 2 |
| **Web Portal Development** | 2025-02-07 | 2025-02-21 | 2 |
| **ML Model - Text Matching** | 2025-02-21 | 2025-03-07 | 2 |
| **ML Model - Image Matching** | 2025-03-07 | 2025-03-21 | 2 |
| **System Integration & Testing** | 2025-03-21 | 2025-04-04 | 2 |
| **Deployment, Security & Final Testing** | 2025-04-04 | 2025-04-11 | 1 |



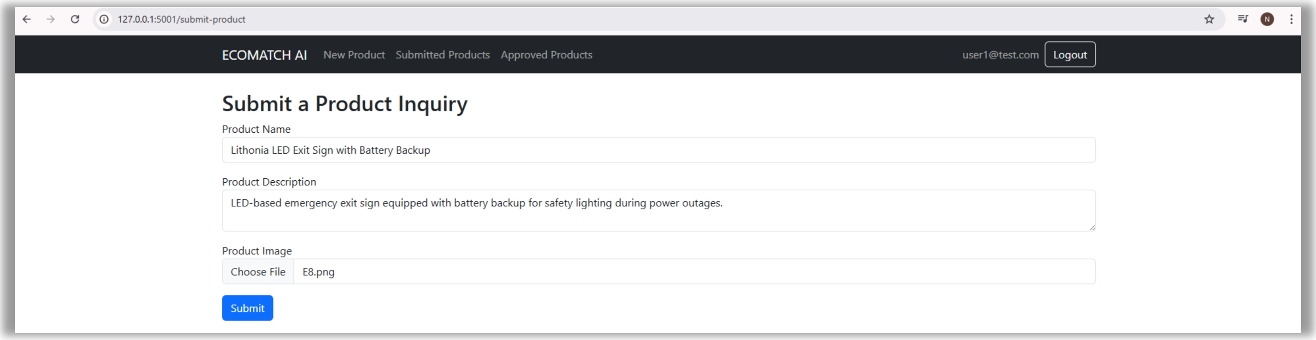
# Implemented Features

## Frontend – UI (member portal)

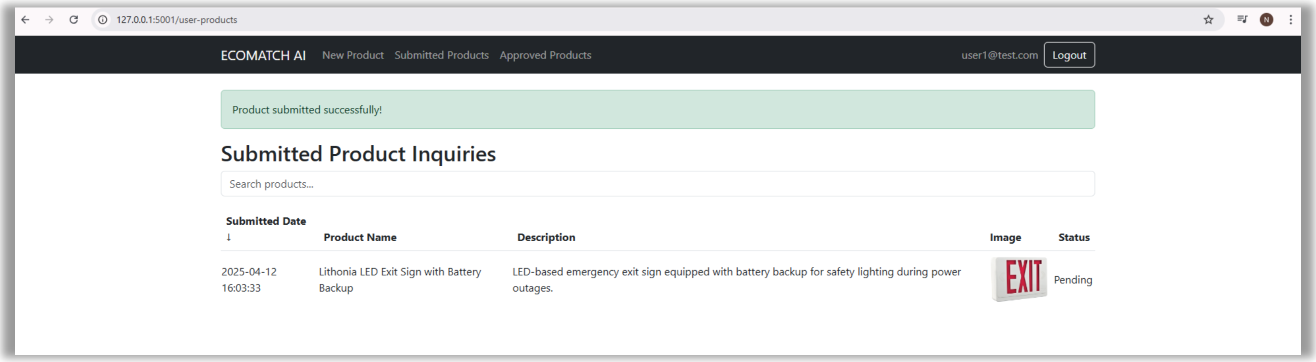
### Member Login Page



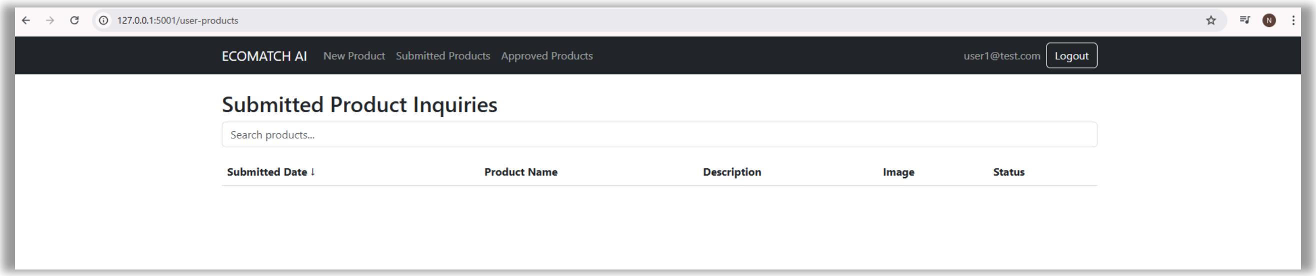

### New Product Page



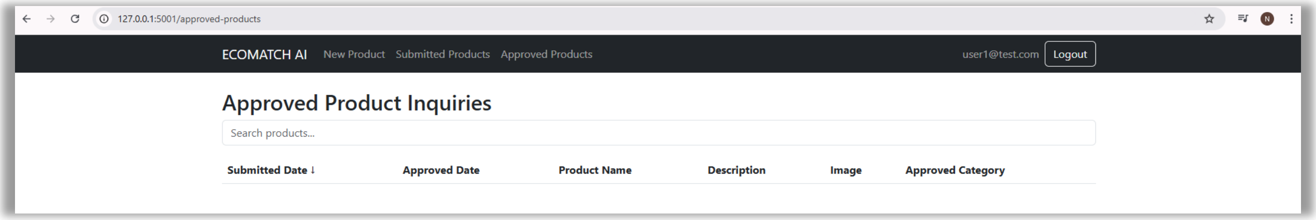

### Submitted Products Page – before approval



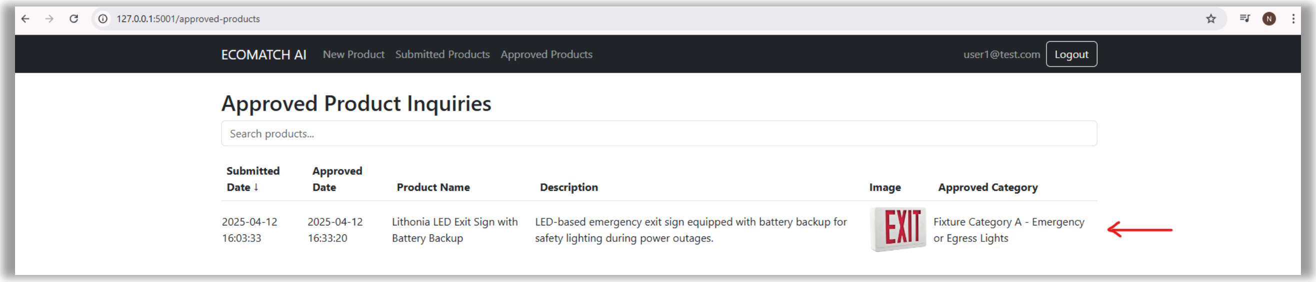

### Submitted Products Page – after approval



### Approved Products Page – before approval

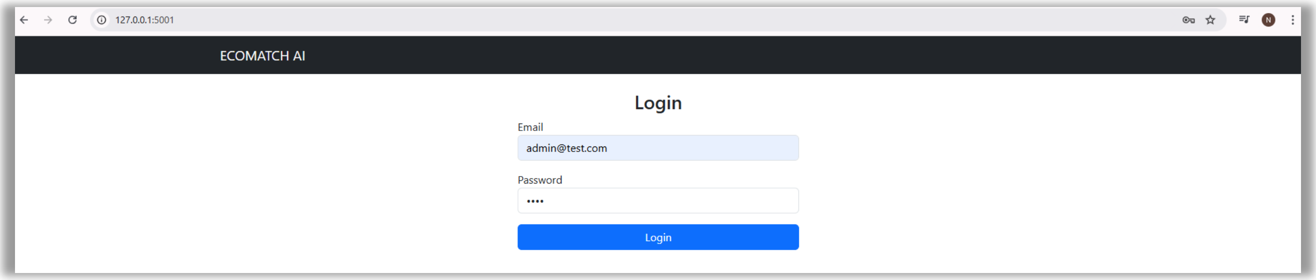



### Approved Products Page – after approval



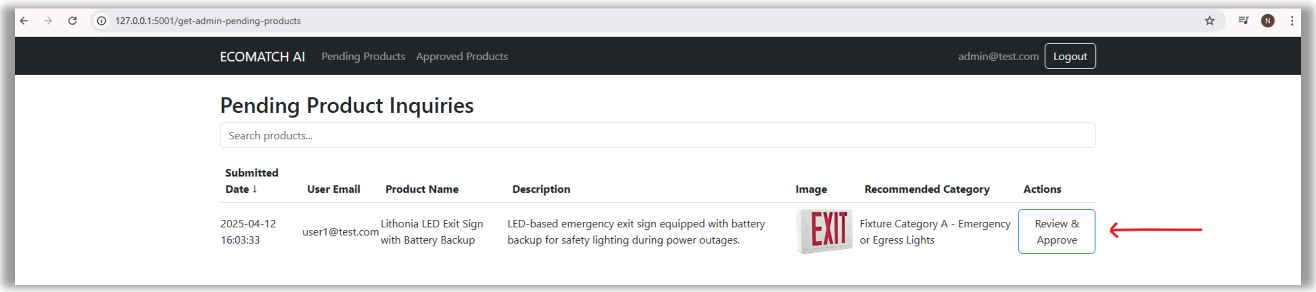

## Frontend – UI (admin portal)

### Admin Login Page

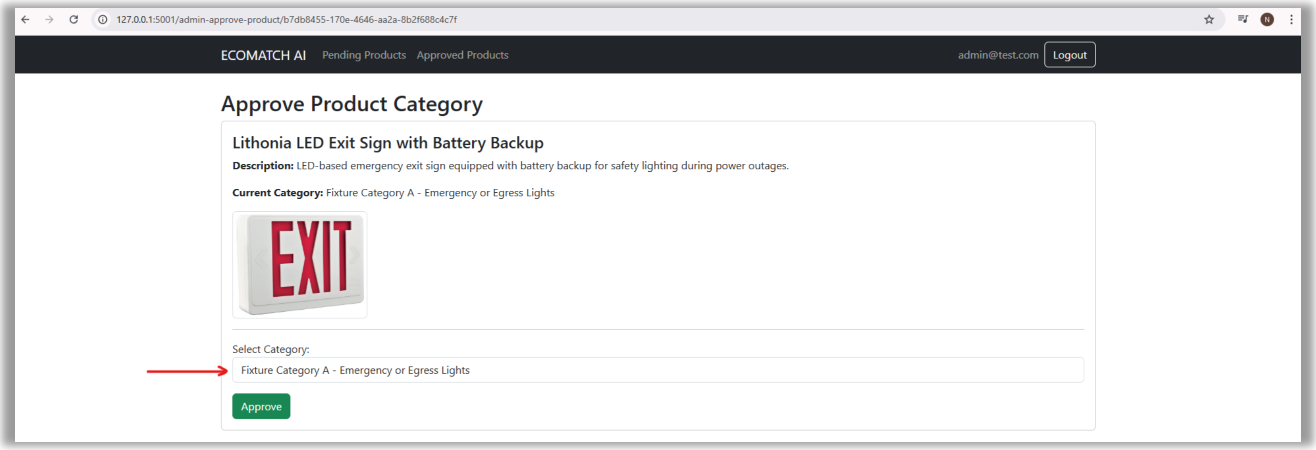



### Admin Landing Page

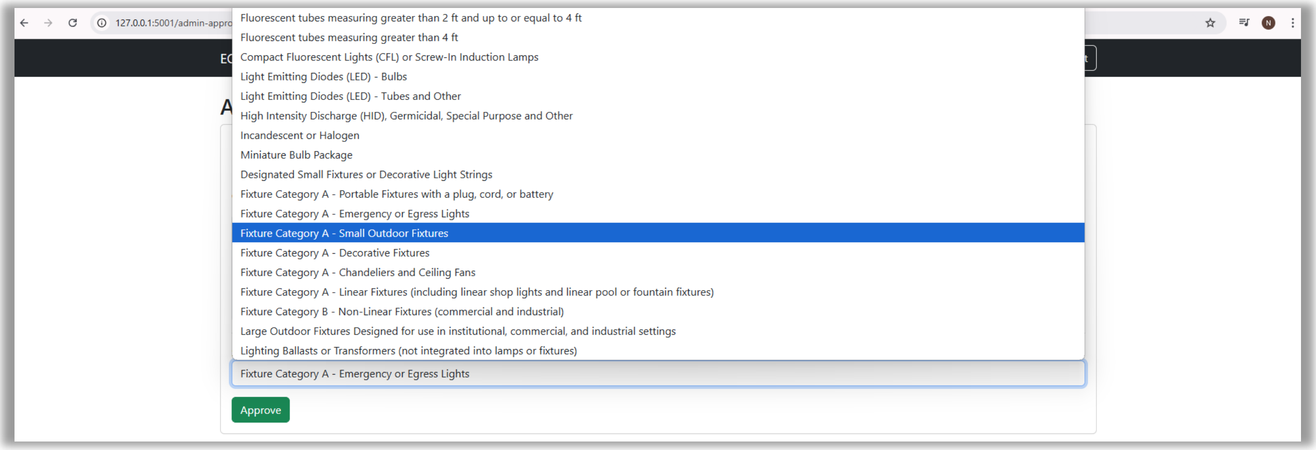

### Pending Products Page

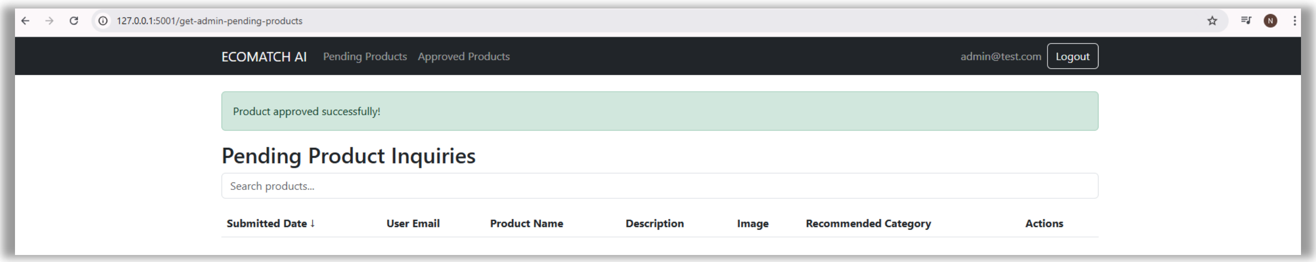


Approve Product Page

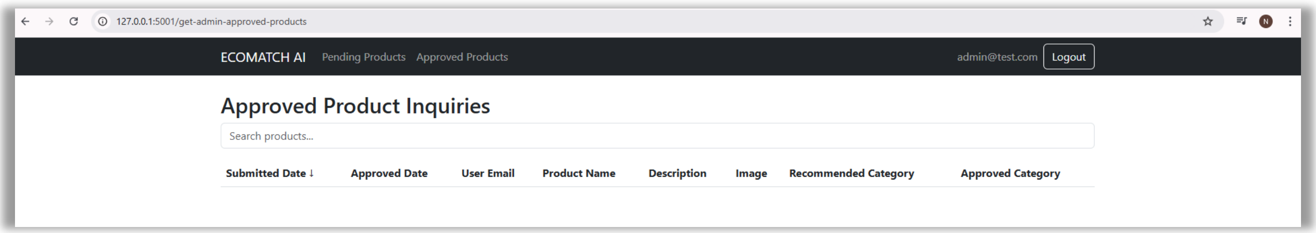



Look (above image) at how the best-matched category has been already selected in the Select Category drop down list. If the admin is not satisfied with the ML model's suggestion, he/she can override it by manually selecting the category from the drop down (below image) and then approve.

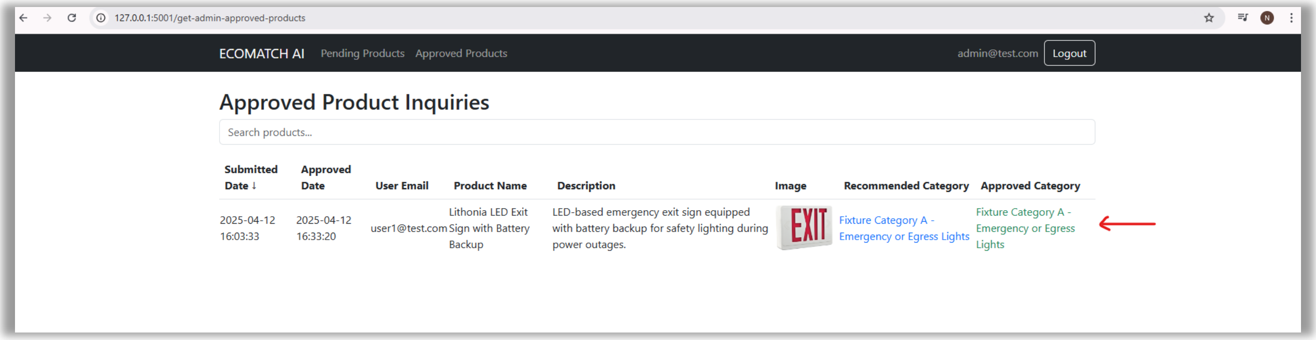



Approved Products Page – before approval



Approved Products Page – after approval



127.0.0.1:5001/get-admin-approved-products

ECOMATCH AI | Pending Products | Approved Products | admin@test.com | Logout

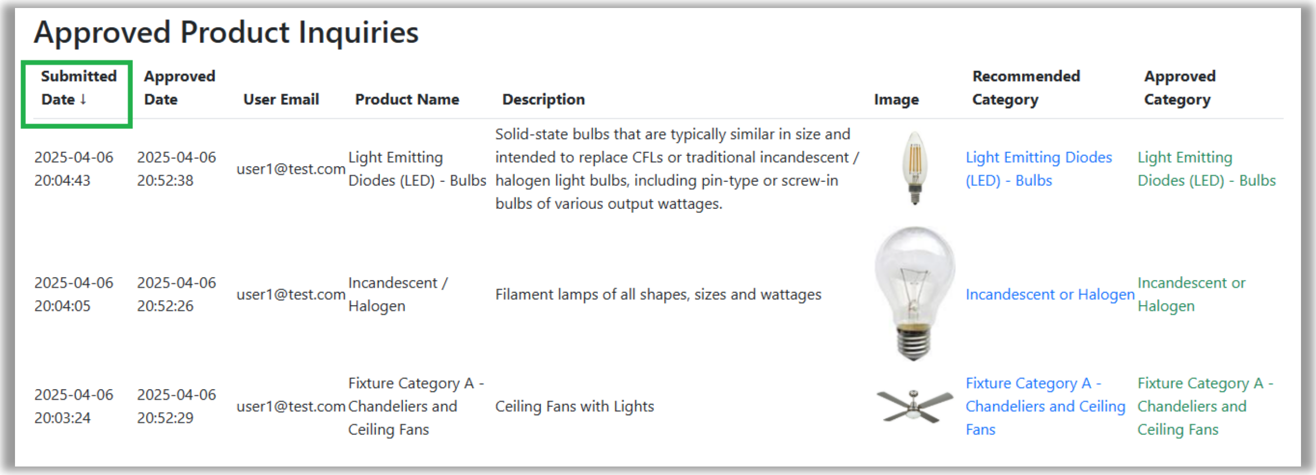
Approved Product Inquiries

Search products...

| Submitted Date ↓ | Approved Date | User Email | Product Name | Description | Image | Recommended Category | Approved Category |
|---|---|---|---|---|---|---|---|
| 2025-04-12 16:03:33 | 2025-04-12 16:33:20 | user1@test.com | Lithonia LED Exit Sign with Battery Backup | LED-based emergency exit sign equipped with battery backup for safety lighting during power outages. | EXIT | Fixture Category A - Emergency or Egress Lights | Fixture Category A - Emergency or Egress Lights |

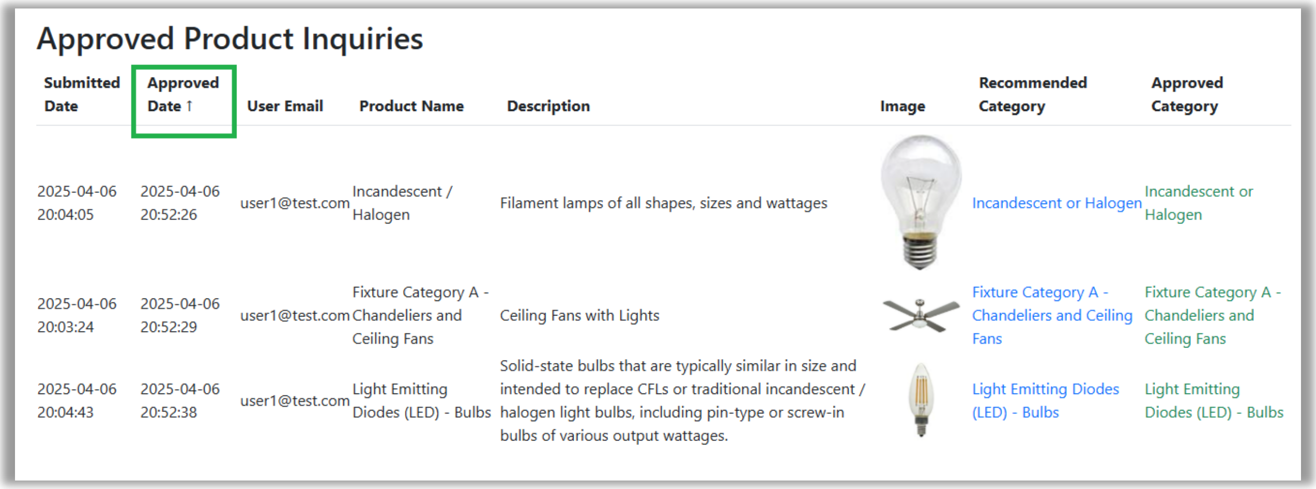
## Sorting Function – Frontend – UI (both member and admin portals)

The sorting functionality in the Approved Products Inquiries (Admin) table is based on the fields – *Submitted Date, Approved Date, User Email, and Product Name*. This same sorting feature has been implemented across all four user interfaces with different fields.



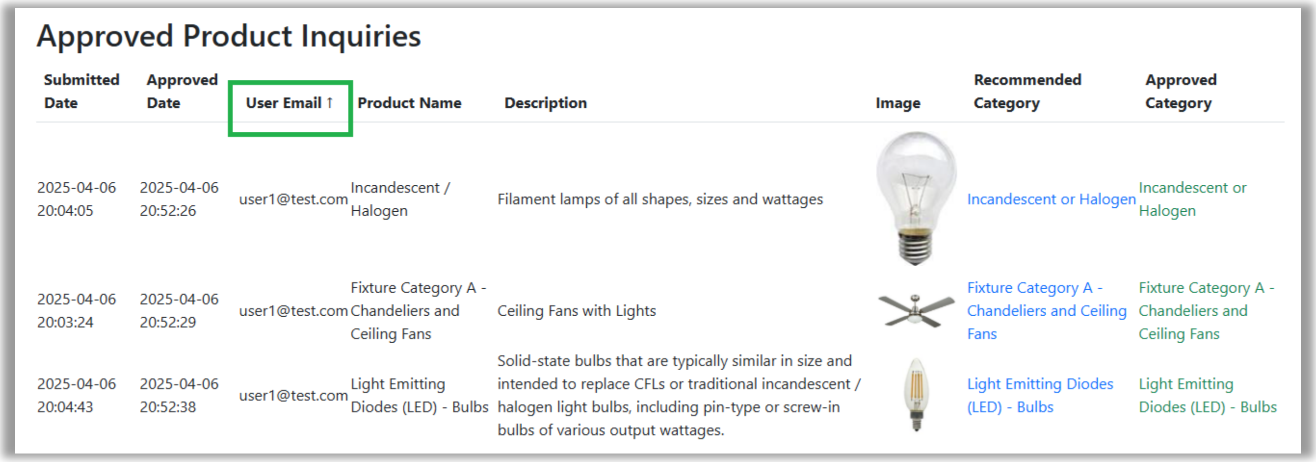
Approved Product Inquiries

| Submitted Date ↓ | Approved Date | User Email | Product Name | Description | Image | Recommended Category | Approved Category |
|---|---|---|---|---|---|---|---|
| 2025-04-06 20:04:43 | 2025-04-06 20:52:38 | user1@test.com | Light Emitting Diodes (LED) - Bulbs | Solid-state bulbs that are typically similar in size and intended to replace CFLs or traditional incandescent / halogen light bulbs, including pin-type or screw-in bulbs of various output wattages. | | Light Emitting Diodes (LED) - Bulbs | Light Emitting Diodes (LED) - Bulbs |
| 2025-04-06 20:04:05 | 2025-04-06 20:52:26 | user1@test.com | Incandescent / Halogen | Filament lamps of all shapes, sizes and wattages | | Incandescent or Halogen | Incandescent or Halogen |
| 2025-04-06 20:03:24 | 2025-04-06 20:52:29 | user1@test.com | Fixture Category A - Chandeliers and Ceiling Fans | Ceiling Fans with Lights | | Fixture Category A - Chandeliers and Ceiling Fans | Fixture Category A - Chandeliers and Ceiling Fans |



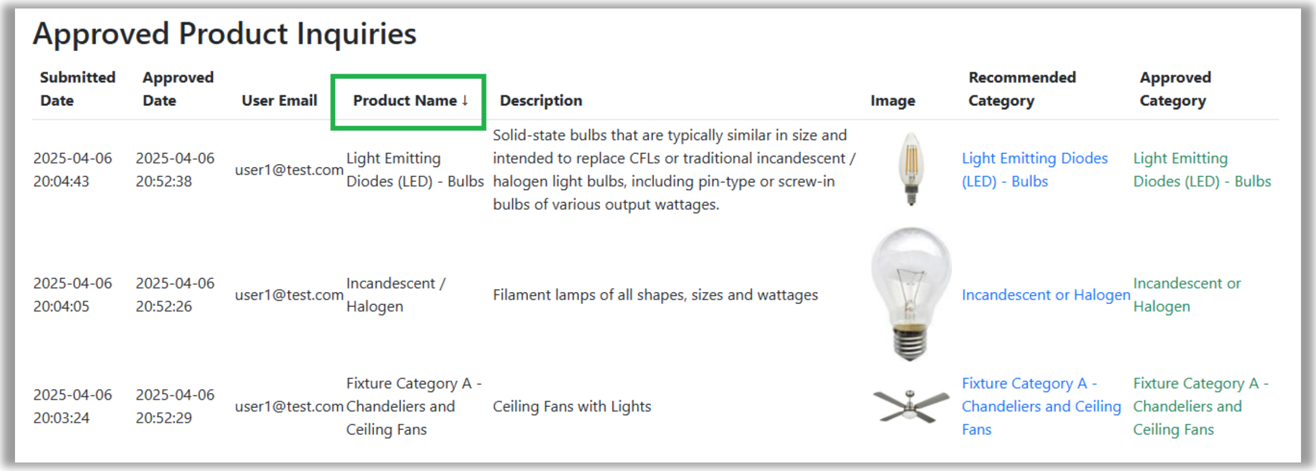
Approved Product Inquiries

| Submitted Date | Approved Date ↑ | User Email | Product Name | Description | Image | Recommended Category | Approved Category |
|---|---|---|---|---|---|---|---|
| 2025-04-06 20:04:05 | 2025-04-06 20:52:26 | user1@test.com | Incandescent / Halogen | Filament lamps of all shapes, sizes and wattages | | Incandescent or Halogen | Incandescent or Halogen |
| 2025-04-06 20:03:24 | 2025-04-06 20:52:29 | user1@test.com | Fixture Category A - Chandeliers and Ceiling Fans | Ceiling Fans with Lights | | Fixture Category A - Chandeliers and Ceiling Fans | Fixture Category A - Chandeliers and Ceiling Fans |
| 2025-04-06 20:04:43 | 2025-04-06 20:52:38 | user1@test.com | Light Emitting Diodes (LED) - Bulbs | Solid-state bulbs that are typically similar in size and intended to replace CFLs or traditional incandescent / halogen light bulbs, including pin-type or screw-in bulbs of various output wattages. | | Light Emitting Diodes (LED) - Bulbs | Light Emitting Diodes (LED) - Bulbs |



## Approved Product Inquiries

| Submitted Date | Approved Date | User Email ↑ | Product Name | Description | Image | Recommended Category | Approved Category |
|---|---|---|---|---|---|---|---|
| 2025-04-06 20:04:05 | 2025-04-06 20:52:26 | user1@test.com | Incandescent / Halogen | Filament lamps of all shapes, sizes and wattages | | Incandescent or Halogen | Incandescent or Halogen |
| 2025-04-06 20:03:24 | 2025-04-06 20:52:29 | user1@test.com | Fixture Category A - Chandeliers and Ceiling Fans | Ceiling Fans with Lights | | Fixture Category A - Chandeliers and Ceiling Fans | Fixture Category A - Chandeliers and Ceiling Fans |
| 2025-04-06 20:04:43 | 2025-04-06 20:52:38 | user1@test.com | Light Emitting Diodes (LED) - Bulbs | Solid-state bulbs that are typically similar in size and intended to replace CFLs or traditional incandescent / halogen light bulbs, including pin-type or screw-in bulbs of various output wattages. | | Light Emitting Diodes (LED) - Bulbs | Light Emitting Diodes (LED) - Bulbs |

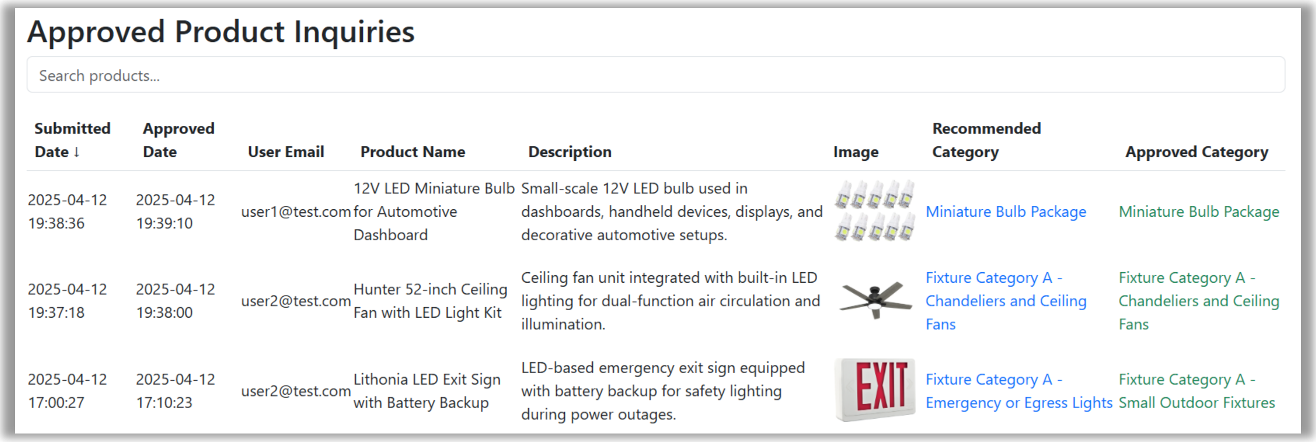


## Approved Product Inquiries

| Submitted Date | Approved Date | User Email | Product Name ↓ | Description | Image | Recommended Category | Approved Category |
|---|---|---|---|---|---|---|---|
| 2025-04-06 20:04:43 | 2025-04-06 20:52:38 | user1@test.com | Light Emitting Diodes (LED) - Bulbs | Solid-state bulbs that are typically similar in size and intended to replace CFLs or traditional incandescent / halogen light bulbs, including pin-type or screw-in bulbs of various output wattages. | | Light Emitting Diodes (LED) - Bulbs | Light Emitting Diodes (LED) - Bulbs |
| 2025-04-06 20:04:05 | 2025-04-06 20:52:26 | user1@test.com | Incandescent / Halogen | Filament lamps of all shapes, sizes and wattages | | Incandescent or Halogen | Incandescent or Halogen |
| 2025-04-06 20:03:24 | 2025-04-06 20:52:29 | user1@test.com | Fixture Category A - Chandeliers and Ceiling Fans | Ceiling Fans with Lights | | Fixture Category A - Chandeliers and Ceiling Fans | Fixture Category A - Chandeliers and Ceiling Fans |

## Search/Filter Function – Frontend – UI (both member and admin portals)

The filtering functionality in the Approved Products Inquiries (Admin) table is based on the fields – Submitted Date, Approved Date, User Email, Product Name, Description, Recommended Category and Approved Category. This feature has been implemented across all 4 user interfaces for different fields.



## Approved Product Inquiries

Search products...

| Submitted Date ↓ | Approved Date | User Email | Product Name | Description | Image | Recommended Category | Approved Category |
|---|---|---|---|---|---|---|---|
| 2025-04-12 19:38:36 | 2025-04-12 19:39:10 | user1@test.com | 12V LED Miniature Bulb for Automotive Dashboard | Small-scale 12V LED bulb used in dashboards, handheld devices, displays, and decorative automotive setups. | | Miniature Bulb Package | Miniature Bulb Package |
| 2025-04-12 19:37:18 | 2025-04-12 19:38:00 | user2@test.com | Hunter 52-inch Ceiling Fan with LED Light Kit | Ceiling fan unit integrated with built-in LED lighting for dual-function air circulation and illumination. | | Fixture Category A - Chandeliers and Ceiling Fans | Fixture Category A - Chandeliers and Ceiling Fans |
| 2025-04-12 17:00:27 | 2025-04-12 17:10:23 | user2@test.com | Lithonia LED Exit Sign with Battery Backup | LED-based emergency exit sign equipped with battery backup for safety lighting during power outages. | EXIT | Fixture Category A - Emergency or Egress Lights | Fixture Category A - Small Outdoor Fixtures |

# Approved Product Inquiries

user2

| Submitted Date ↓ | Approved Date | User Email | Product Name | Description | Image | Recommended Category | Approved Category |
|---|---|---|---|---|---|---|---|
| 2025-04-12 19:37:18 | 2025-04-12 19:38:00 | user2@test.com | Hunter 52-inch Ceiling Fan with LED Light Kit | Ceiling fan unit integrated with built-in LED lighting for dual-function air circulation and illumination. |  | Fixture Category A - Chandeliers and Ceiling Fans | Fixture Category A - Chandeliers and Ceiling Fans |
| 2025-04-12 17:00:27 | 2025-04-12 17:10:23 | user2@test.com | Lithonia LED Exit Sign with Battery Backup | LED-based emergency exit sign equipped with battery backup for safety lighting during power outages. |   | Fixture Category A - Emergency or Egress Lights | Fixture Category A - Small Outdoor Fixtures |

# Approved Product Inquiries

mini

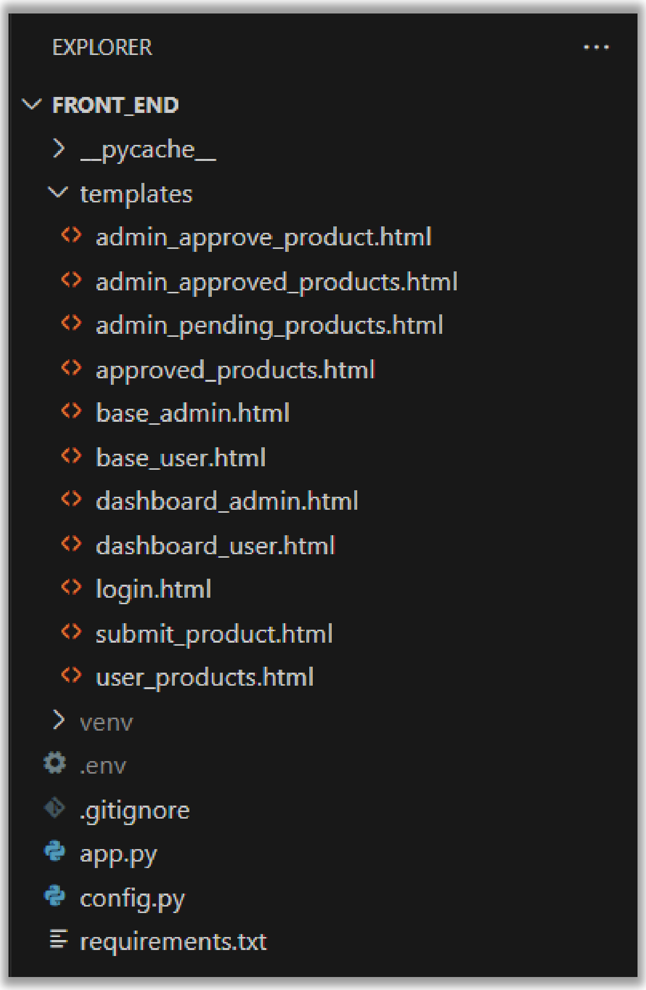
| Submitted Date ↓ | Approved Date | User Email | Product Name | Description | Image | Recommended Category | Approved Category |
|---|---|---|---|---|---|---|---|
| 2025-04-12 19:38:36 | 2025-04-12 19:39:10 | user1@test.com | 12V LED Miniature Bulb for Automotive Dashboard | Small-scale 12V LED bulb used in dashboards, handheld devices, displays, and decorative automotive setups. |  | Miniature Bulb Package | Miniature Bulb Package |

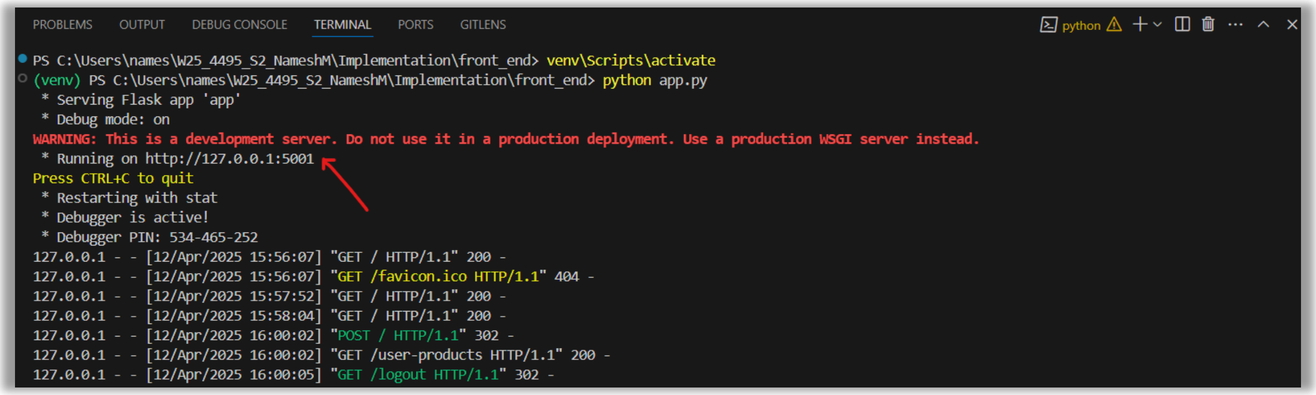
# Approved Product Inquiries

04-12

| Submitted Date ↓ | Approved Date | User Email | Product Name | Description | Image | Recommended Category | Approved Category |
|---|---|---|---|---|---|---|---|
| 2025-04-12 19:38:36 | 2025-04-12 19:39:10 | user1@test.com | 12V LED Miniature Bulb for Automotive Dashboard | Small-scale 12V LED bulb used in dashboards, handheld devices, displays, and decorative automotive setups. |  | Miniature Bulb Package | Miniature Bulb Package |
| 2025-04-12 19:37:18 | 2025-04-12 19:38:00 | user2@test.com | Hunter 52-inch Ceiling Fan with LED Light Kit | Ceiling fan unit integrated with built-in LED lighting for dual-function air circulation and illumination. |  | Fixture Category A - Chandeliers and Ceiling Fans | Fixture Category A - Chandeliers and Ceiling Fans |
| 2025-04-12 17:00:27 | 2025-04-12 17:10:23 | user2@test.com | Lithonia LED Exit Sign with Battery Backup | LED-based emergency exit sign equipped with battery backup for safety lighting during power outages. |   | Fixture Category A - Emergency or Egress Lights | Fixture Category A - Small Outdoor Fixtures |

## Frontend – Code



app.py

```python
from flask import Flask, render_template, request, redirect, url_for, flash, session
from azure.storage.blob import BlobServiceClient
import uuid
import re
import requests
from dotenv import load_dotenv
from config import Config
from azure.storage.blob import generate_blob_sas, BlobSasPermissions
from datetime import datetime, timezone
from dateutil.relativedelta import relativedelta


# Load environment variables from .env file
load_dotenv()

app = Flask(__name__)
app.config.from_object(Config)

# Azure Blob Storage Setup
BLOB_SERVICE_URL = f"https://{Config.AZURE_STORAGE_ACCOUNT}.blob.core.windows.net/"
blob_service_client = BlobServiceClient(account_url=BLOB_SERVICE_URL, credential=Config.AZURE_STORAGE_KEY)

# Removes invalid characters and replaces spaces with hyphens
def sanitize_filename(filename):
    filename = filename.lower().replace(" ", "-")  # Convert spaces to hyphens
    filename = re.sub(r"[^a-zA-Z0-9.\-_]", "", filename)  # Remove invalid characters
    return filename

# Render login form and handle authentication
@app.route("/", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        email = request.form["email"]
        password = request.form["password"]

        # Send request to backend API
        response = requests.post(f"{Config.BACKEND_URL}/login", json={"email": email, "password": password})

        if response.status_code == 200:
            data = response.json()
            session["token"] = data["token"]
            session["email"] = data["email"]
            session["role"] = data["role"]

            # Redirect based on role
            if data["role"] == "admin":
                return redirect(url_for("dashboard_admin"))
```

## Backend – Code

```
EXPLORER                              ...
∨ BACK_END
  > __pycache__
  > venv
    .env
    .gitignore
    add_user.py
    app.py
    config.py
    create_table.py
    requirements.txt
    utils.py
```

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    GITLENS

PS C:\Users\names\W25_4495_S2_NameshM\Implementation\back_end> venv\Scripts\activate
(venv) PS C:\Users\names\W25_4495_S2_NameshM\Implementation\back_end> python app.py
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5000
Press CTRL+C to quit
 * Restarting with stat
 * Debugger is active!
 * Debugger PIN: 821-688-295
127.0.0.1 - - [12/Apr/2025 16:53:18] "POST /login HTTP/1.1" 200 -
127.0.0.1 - - [12/Apr/2025 16:53:18] "GET /get-user-products HTTP/1.1" 200 -
127.0.0.1 - - [12/Apr/2025 16:53:21] "GET /get-user-products HTTP/1.1" 200 -
127.0.0.1 - - [12/Apr/2025 16:53:23] "GET /get-approved-products HTTP/1.1" 200 -
```

create_table.py

```python
# create_table.py ×
# create_table.py > ...
# You, 2 months ago | 1 author (You)
from azure.data.tables import TableServiceClient
from config import Config

# Initialize Table Service Client
table_service = TableServiceClient.from_connection_string(Config.AZURE_STORAGE_CONNECTION_STRING)
table_client = table_service.get_table_client(Config.AZURE_USER_TABLE_NAME)

# Create Table if it doesn't exist
table_client.create_table()
print(f"Table '{Config.AZURE_USER_TABLE_NAME}' is ready.")    You, 2 months ago • Implemented user
```

add-user.py

add_user.py ×

add_user.py > ...

You, 2 months ago | 1 author (You)

```python
import uuid        You, 2 months ago • Implemented user logins of the web portal …
from azure.data.tables import TableServiceClient, TableEntity
from config import Config
import bcrypt

# Azure Table connection
table_service = TableServiceClient.from_connection_string(Config.AZURE_STORAGE_CONNECTION_STRING)
user_table = table_service.get_table_client(Config.AZURE_USER_TABLE_NAME)

# Generate a bcrypt hash for a given password
def hash_password(password):
    return bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")

# Insert a user into Azure Table Storage
def add_user(email, password):
    user_id = str(uuid.uuid4())  # Generate unique ID
    hashed_password = hash_password(password)

    user_entity = TableEntity()
    user_entity["PartitionKey"] = "users"  # Fixed partition key for all users
    user_entity["RowKey"] = user_id  # Unique identifier for each user
    user_entity["Email"] = email
    user_entity["PasswordHash"] = hashed_password

    user_table.create_entity(user_entity)
    print(f"User {email} added successfully!")

# Run this script with example user
if __name__ == "__main__":
    email = input("Enter email: ")
    password = input("Enter password: ")
    add_user(email, password)
```

utils.py

utils.py ×

utils.py > verify_password

You, 2 months ago | 1 author (You)

```python
import bcrypt

# Generate a bcrypt hash for a given password
def hash_password(password):
    return bcrypt.hashpw(password.encode("utf-8"), bcrypt.gensalt()).decode("utf-8")

# Verify a password against a stored hash
def verify_password(stored_password, provided_password):
    return bcrypt.checkpw(provided_password.encode("utf-8"), stored_password.encode("utf-8"))
```

app.py

Help back_end

app.py

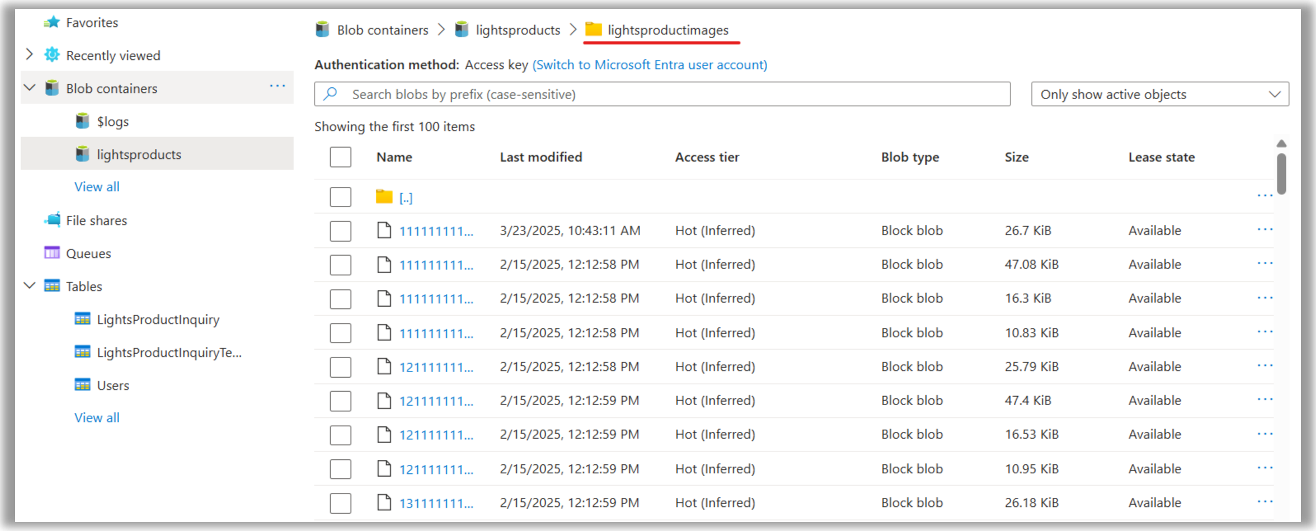
app.py > ...

You, 3 days ago | 1 author (You)

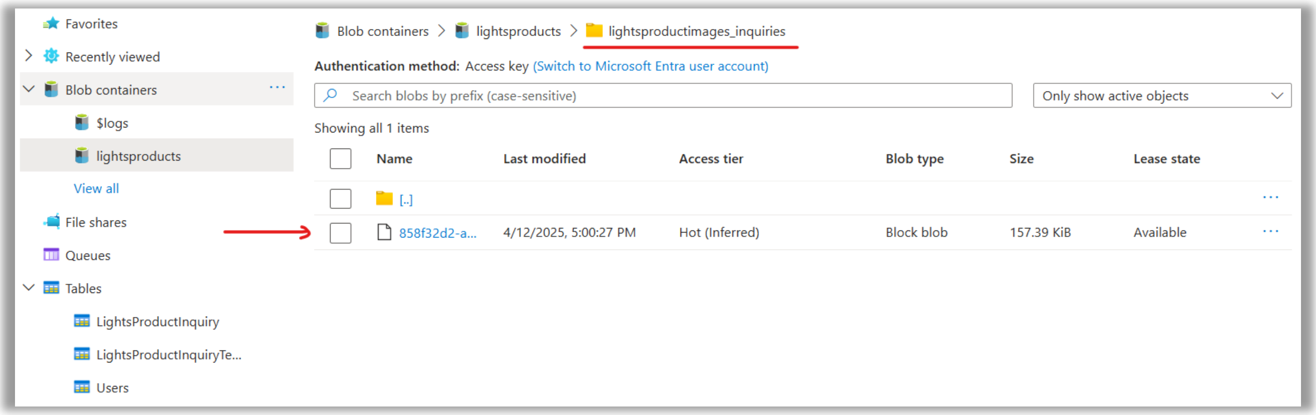
```python
from flask import Flask, request, jsonify
from azure.data.tables import TableServiceClient, TableEntity
from flask_jwt_extended import JWTManager, jwt_required, get_jwt_identity, create_access_token, get_jwt
from werkzeug.security import check_password_hash
from datetime import datetime
import uuid
import requests
import pprint
        You, 2 months ago • Implemented user logins of the web portal …
from config import Config
from utils import verify_password

app = Flask(__name__)
app.config.from_object(Config)  # Load configuration from config.py

jwt = JWTManager(app)

# Azure Table Storage connection
table_service = TableServiceClient.from_connection_string(Config.AZURE_STORAGE_CONNECTION_STRING)
user_table = table_service.get_table_client(Config.AZURE_USER_TABLE_NAME)
product_table = table_service.get_table_client(Config.AZURE_TEMP_PRODUCT_TABLE_NAME)

# Handle user login
@app.route("/login", methods=["POST"])
def login():
    data = request.get_json()
    email = data.get("email")
    password = data.get("password")

    if not email or not password:
        return jsonify({"error": "Email and password are required"}), 400

    # Query user by email
    try:
        query = f"Email eq '{email}'"
        users = list(user_table.query_entities(query))

        if not users:
            return jsonify({"error": "Invalid email or password"}), 401

        user = users[0]  # First matching user
        stored_password = user["PasswordHash"]

        # Verify password
        if not verify_password(stored_password, password):
            return jsonify({"error": "Invalid email or password"}), 401

```

## Azure Blob Storage and Azure Table Storage

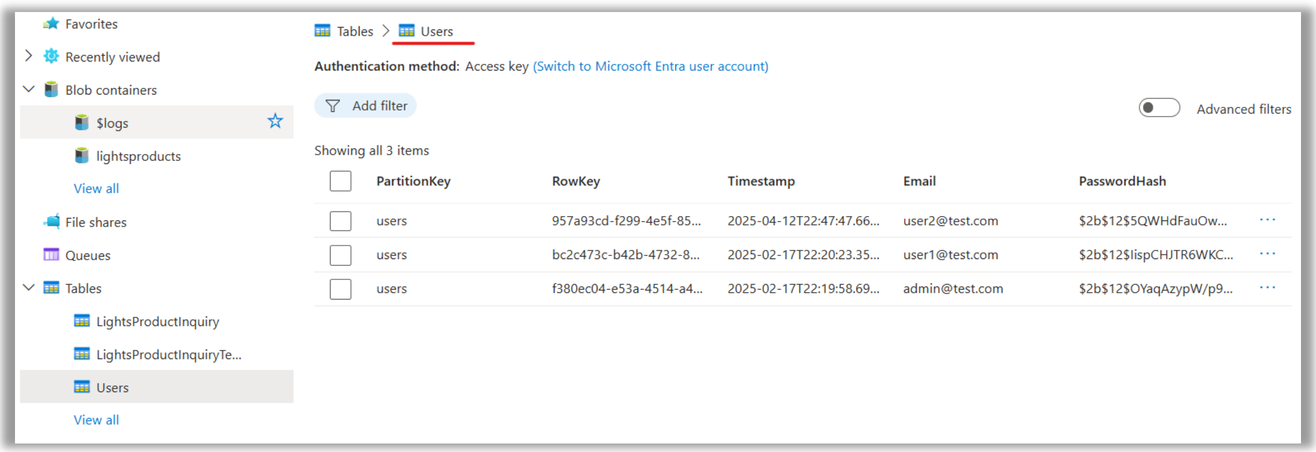
lightsproductimages – Azure Blob Storage that stores the images of the products in the product guide



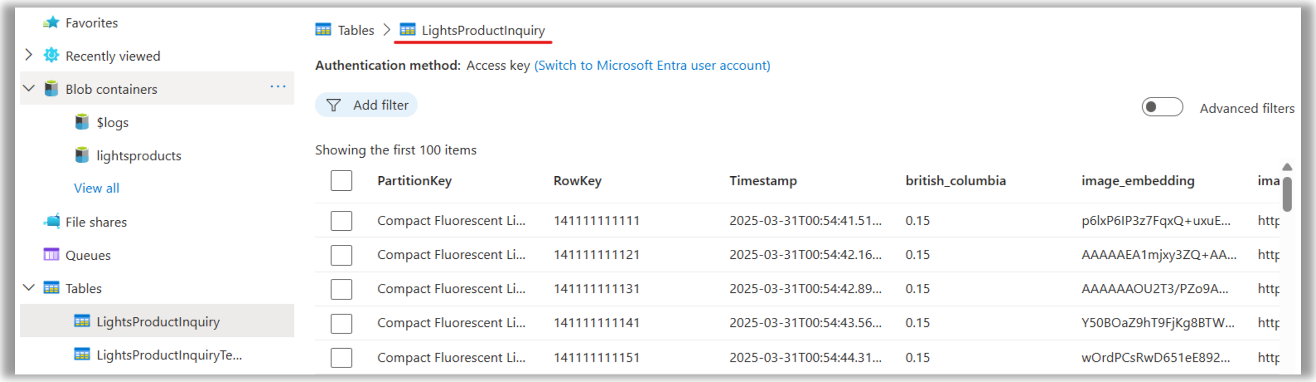

lightsproductimages_inquiries – Azure Blob Storage that stores the images of the products in the new product inquiries



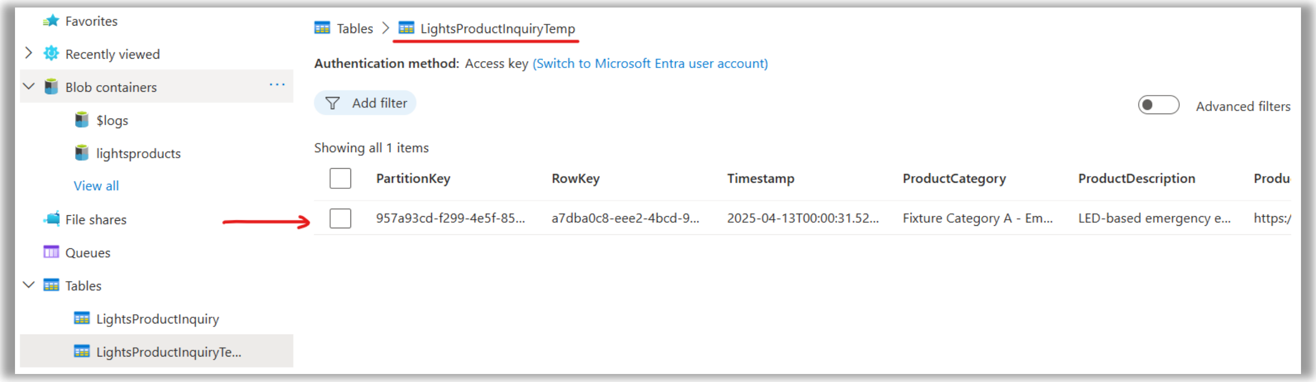

Users - Azure Table that stores the details of the users of the web portal



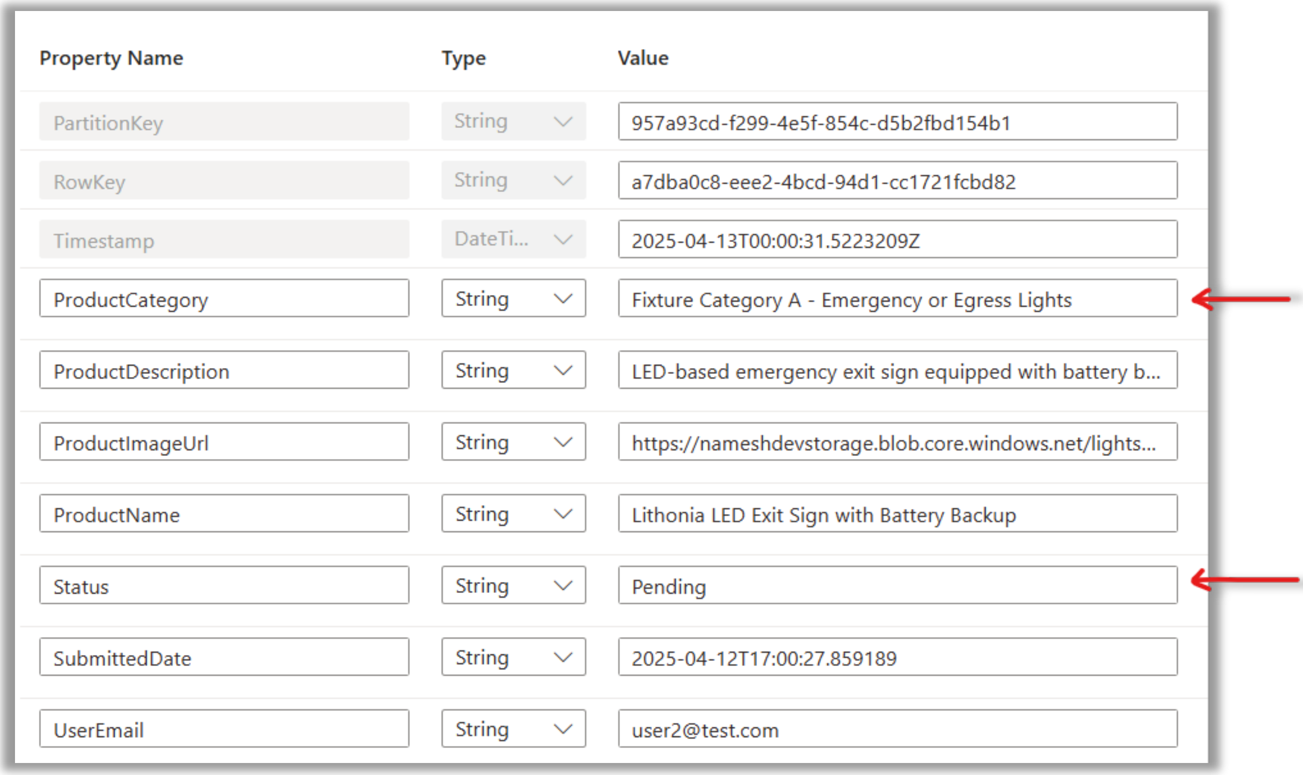
LightsProductInquiry - Azure Table that stores the textual data of the products in the product guide




LightsProductInquiryTemp - Azure Table that stores the textual data of the new product inquiries




Newly submitted product inquiry in LightsProductInquiryTemp – before approval



Newly submitted product inquiry in LightsProductInquiryTemp – after approval

| Property Name | Type | Value |
| --- | --- | --- |
| PartitionKey | String | 957a93cd-f299-4e5f-854c-d5b2fbd154b1 |
| RowKey | String | a7dba0c8-eee2-4bcd-94d1-cc1721fcbd82 |
| Timestamp | DateTi... | 2025-04-13T00:10:22.6965856Z |
| ApprovedDate | String | 2025-04-12T17:10:23.246182 |
| ApprovedProductCategory | String | Fixture Category A - Small Outdoor Fixtures |
| ProductCategory | String | Fixture Category A - Emergency or Egress Lights |
| ProductDescription | String | LED-based emergency exit sign equipped with battery b... |
| ProductImageUrl | String | https://nameshdevstorage.blob.core.windows.net/lights... |
| ProductName | String | Lithonia LED Exit Sign with Battery Backup |
| Status | String | Approved |
| SubmittedDate | String | 2025-04-12T17:00:27.859189 |
| UserEmail | String | user2@test.com |

## ML Model – Code

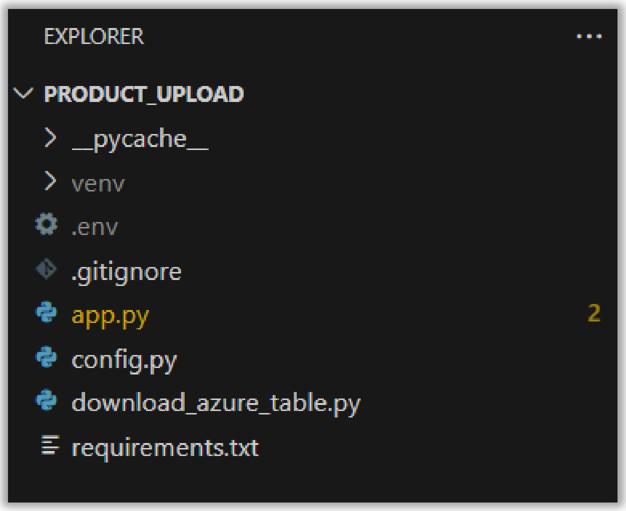
```
EXPLORER ...
ML_MODEL
  > __pycache__
  > venv
  .env
  .gitignore
  app.py                3
  config.py
  model_evaluation_pc.py
  model_evaluation_pd.py
  old.txt
  requirements.txt
```

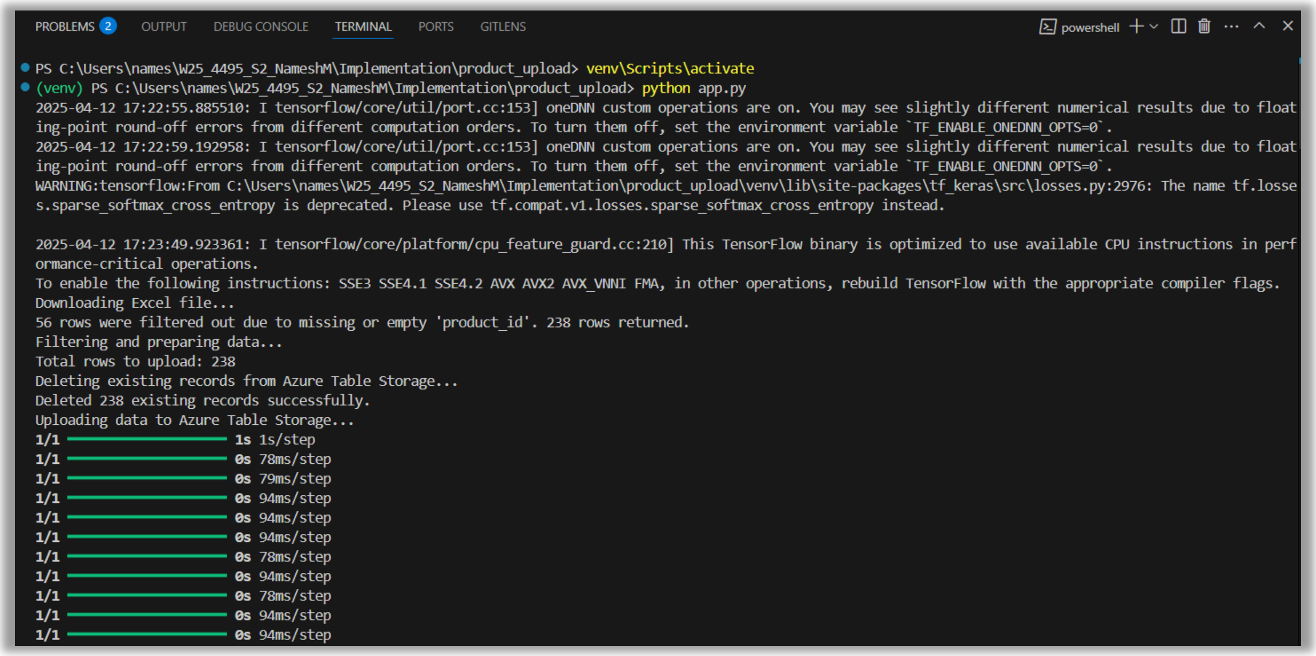
```
PROBLEMS 3   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS                python

PS C:\Users\names\W25_4495_S2_NameshM\Implementation\ml_model> venv\Scripts\activate
(venv) PS C:\Users\names\W25_4495_S2_NameshM\Implementation\ml_model> python app.py
2025-04-12 16:56:52.934557: I tensorflow/core/platform/cpu_feature_guard.cc:182] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
To enable the following instructions: SSE SSE2 SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
 * Serving Flask app 'app'
 * Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
 * Running on http://127.0.0.1:5002
Press CTRL+C to quit
 * Restarting with stat
2025-04-12 16:57:05.000889: I tensorflow/core/platform/cpu_feature_guard.cc:182] This TensorFlow binary is optimized to use available CPU instructions in performance-critical operations.
 flags.
 * Debugger is active!
 * Debugger PIN: 141-561-298
1/1 [==============================] - 1s 1s/step
Based on Product Category:  Fixture Category A - Emergency or Egress Lights    |    33.08459222316742
Based on Product Description:  Fixture Category A - Emergency or Egress Lights    |    56.13521933555603
Based on Product Image:  Fixture Category A - Emergency or Egress Lights    |    58.09372067451477

Final Match:
product_category: Fixture Category A - Emergency or Egress Lights
product_id: 201131111121
product_description: Emergency / Egress Lights
row_key: 201131111121

Final Score: 58.09372067451477

127.0.0.1 - - [12/Apr/2025 17:00:32] "POST /predict-category HTTP/1.1" 200 -
```

app.py

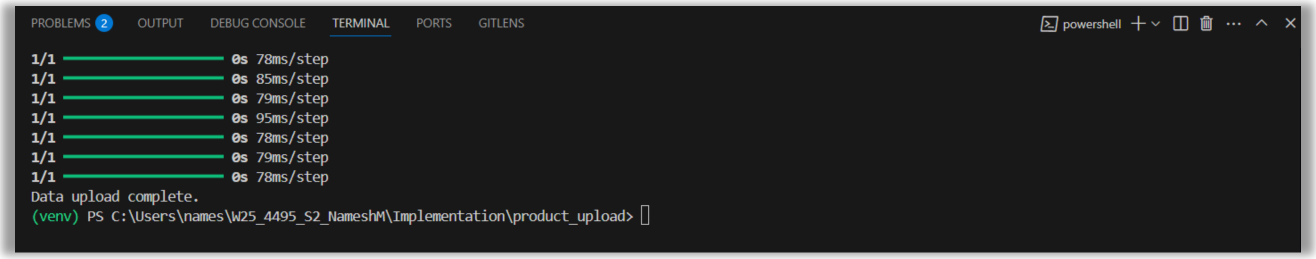
```python
Help                                ml_model
app.py 3 ×
app.py > ...
You, 3 days ago | 1 author (You)
from flask import Flask, request, jsonify
from azure.data.tables import TableServiceClient
from sentence_transformers import SentenceTransformer
import numpy as np
from config import Config
from azure.core.credentials import AzureNamedKeyCredential

import requests
from PIL import Image
from io import BytesIO
from tensorflow.keras.applications.resnet50 import ResNet50, preprocess_input
from tensorflow.keras.preprocessing import image
from tensorflow.keras.models import Model
import base64
        You, 2 months ago • Implemented ML model API and connection to back…
app = Flask(__name__)
app.config.from_object(Config)

# Create credential object
credential = AzureNamedKeyCredential(Config.AZURE_STORAGE_ACCOUNT_NAME, Config.AZURE_STORAGE_ACCOUNT_KEY)

# Azure Storage URL
TABLE_SERVICE_URL = f"https://{Config.AZURE_STORAGE_ACCOUNT_NAME}.table.core.windows.net/"

# Load models
model_pc = SentenceTransformer('all-mpnet-base-v2')
model_pd = SentenceTransformer('bert-large-nli-stsb-mean-tokens')
model_img = ResNet50(weights='imagenet', include_top=False, pooling='avg')


def decode_array(encoded_string):
    decoded = base64.b64decode(encoded_string)
    return np.frombuffer(decoded, dtype=np.float32)


def fetch_products_with_embeddings():
    try:
        service = TableServiceClient(endpoint=TABLE_SERVICE_URL, credential=credential)
        table_client = service.get_table_client(table_name=Config.AZURE_TABLE_NAME)

        products = []
        for entity in table_client.list_entities():
            products.append({
```

## Product Upload – Code




```
PS C:\Users\names\W25_4495_S2_NameshM\Implementation\product_upload> venv\Scripts\activate
(venv) PS C:\Users\names\W25_4495_S2_NameshM\Implementation\product_upload> python app.py
2025-04-12 17:22:55.885510: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to float
ing-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
2025-04-12 17:22:59.192958: I tensorflow/core/util/port.cc:153] oneDNN custom operations are on. You may see slightly different numerical results due to float
ing-point round-off errors from different computation orders. To turn them off, set the environment variable `TF_ENABLE_ONEDNN_OPTS=0`.
WARNING:tensorflow:From C:\Users\names\W25_4495_S2_NameshM\Implementation\product_upload\venv\lib\site-packages\tf_keras\src\losses.py:2976: The name tf.losse
s.sparse_softmax_cross_entropy is deprecated. Please use tf.compat.v1.losses.sparse_softmax_cross_entropy instead.

2025-04-12 17:23:49.923361: I tensorflow/core/platform/cpu_feature_guard.cc:210] This TensorFlow binary is optimized to use available CPU instructions in perf
ormance-critical operations.
To enable the following instructions: SSE3 SSE4.1 SSE4.2 AVX AVX2 AVX_VNNI FMA, in other operations, rebuild TensorFlow with the appropriate compiler flags.
Downloading Excel file...
56 rows were filtered out due to missing or empty 'product_id'. 238 rows returned.
Filtering and preparing data...
Total rows to upload: 238
Deleting existing records from Azure Table Storage...
Deleted 238 existing records successfully.
Uploading data to Azure Table Storage...
1/1 ━━━━━━━━━━━━━━━━━━━━ 1s 1s/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 79ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 94ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 94ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 94ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 94ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 94ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 94ms/step
```




```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 85ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 79ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 95ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 79ms/step
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 78ms/step
Data upload complete.
(venv) PS C:\Users\names\W25_4495_S2_NameshM\Implementation\product_upload>
```

app.py

```python
import io
import pandas as pd
import uuid
from azure.storage.blob import BlobServiceClient, generate_blob_sas, BlobSasPermissions
from azure.data.tables import TableServiceClient
from azure.core.credentials import AzureNamedKeyCredential
from datetime import datetime, timezone
from dateutil.relativedelta import relativedelta
from flask import Flask
import numpy as np
import base64
import requests
from PIL import Image
from tensorflow.keras.applications.resnet50 import ResNet50, preprocess_input
from tensorflow.keras.preprocessing import image
from sentence_transformers import SentenceTransformer
from config import config
from io import BytesIO


app = Flask(__name__)
app.config.from_object(config)

# Create credential object
credential = AzureNamedKeyCredential(config.AZURE_STORAGE_ACCOUNT_NAME, config.AZURE_STORAGE_ACCOUNT_KEY)

# Azure Storage URLs
BLOB_SERVICE_URL = f"https://{config.AZURE_STORAGE_ACCOUNT_NAME}.blob.core.windows.net/"
TABLE_SERVICE_URL = f"https://{config.AZURE_STORAGE_ACCOUNT_NAME}.table.core.windows.net/"

# Initialize Azure clients
blob_service_client = BlobServiceClient(account_url=BLOB_SERVICE_URL, credential=config.AZURE_STORAGE_ACCOUNT_KEY)
table_service_client = TableServiceClient(endpoint=TABLE_SERVICE_URL, credential=credential)
table_client = table_service_client.get_table_client(table_name=config.AZURE_TABLE_NAME)
```

# Evaluation Techniques

## Model Selection for Text Similarity

All models were assessed using a sample dataset containing product information. Each new product entry included a Product Name and Product Description, referred to as query sentences. These new products were compared against existing product records, which contained a Product Category and Product Description, referred to as comparison sentences.

model_evaluation_pc.py M ×   model_evaluation_pd.py M

model_evaluation_pc.py > ...

```python
# New product names
new_product_name = [
    "T8 18-inch Fluorescent Tube Light",
    "T5 3-ft Fluorescent Tube for Office Lighting",
    "T12 5-ft Fluorescent Tube for Warehouse Use",
    "Philips 13W Spiral CFL Bulb",
    "GE 10W LED A19 Bulb",
    "T8 4-ft LED Tube Light for Commercial Use",
    "Metal Halide 400W HID Lamp for Stadium Lighting",
    "GE 43W Halogen A19 Bulb",
    "12V LED Miniature Bulb for Automotive Dashboard",
    "LED Warm White String Lights for Holiday Decoration",
    "Rechargeable LED Work Light with Handle",
    "Lithonia LED Exit Sign with Battery Backup",
    "Solar-Powered LED Wall Lantern for Outdoor Use",
    "Crystal Chandelier Pendant Light for Dining Room",
    "Hunter 52-inch Ceiling Fan with LED Light Kit",
    "4-ft LED Shop Light for Garages",
    "High Bay LED Light for Warehouse Applications",
    "Streetlight LED Fixture for Highway Lighting",
    "Advance T8 Electronic Ballast for Fluorescent Tubes"
]

# Ground truth mapping (expected indices of correct matches)
true_indices = list(range(len(new_product_name)))

# Function to evaluate a model using Top-K Accuracy and MRR
def evaluate_model(model_name, k=5):
    print(f"Evaluating Model: {model_name} with k={k}")
```

PROBLEMS   OUTPUT   DEBUG CONSOLE   TERMINAL   PORTS   GITLENS

```
Evaluating Model: all-mpnet-base-v2 with k=1
Evaluating Model: bert-base-nli-mean-tokens with k=1
Evaluating Model: bert-large-nli-stsb-mean-tokens with k=1
Evaluating Model: distilbert-base-nli-stsb-mean-tokens with k=1
Evaluating Model: roberta-base-nli-stsb-mean-tokens with k=1
Evaluating Model: roberta-large-nli-stsb-mean-tokens with k=1
Evaluating Model: paraphrase-multilingual-MiniLM-L12-v2 with k=1
                             Model Name  Top-K Accuracy  Mean Reciprocal Rank (MRR)
0                      all-MiniLM-L6-v2        0.421053                    0.612155
1                     all-MiniLM-L12-v2        0.473684                    0.620081
2                     all-mpnet-base-v2        0.578947                    0.708897
3             bert-base-nli-mean-tokens        0.263158                    0.442194
4       bert-large-nli-stsb-mean-tokens        0.368421                    0.478592
5  distilbert-base-nli-stsb-mean-tokens        0.052632                    0.305754
6     roberta-base-nli-stsb-mean-tokens        0.263158                    0.380352
7    roberta-large-nli-stsb-mean-tokens        0.263158                    0.436763
8 paraphrase-multilingual-MiniLM-L12-v2        0.421053                    0.518903
```

The matching process was based on the following criteria:

- **Criteria 1**: The Product Name of the new product was matched against the Product Category of the existing product in the product guide ***(shorter text)***.
- **Criteria 2**: The Product Description of the new product was matched against the Product Description of the existing product in the product guide ***(longer text)***.

All 9 models were evaluated based on these two matching criteria. The results are presented below.

In our case,

- MRR helps check if the correct product match appears as the **first result**.
- Top-K Accuracy helps determine if the right product appears **within the top K results**, ensuring web portal admin gets relevant suggestions.

Since our primary goal is to retrieve the correct product match as the top result, **Mean Reciprocal Rank (MRR) and Top-1 Accuracy are the most suitable metrics** for evaluating our models.

**Criteria 1 – Shorter Text (Product Name)**

| Model Name | Mean Reciprocal Rank (MRR) | Top-1 Accuracy | Top-3 Accuracy | Top-5 Accuracy |
|---|---|---|---|---|
| all-mpnet-base-v2 | 0.7089 | 0.5789 | 0.7895 | 0.8421 |
| all-MiniLM-L12-v2 | 0.6201 | 0.4737 | 0.7368 | 0.7895 |
| all-MiniLM-L6-v2 | 0.6122 | 0.4211 | 0.7895 | 0.8421 |
| paraphrase-multilingual-MiniLM-L12-v2 | 0.5189 | 0.4211 | 0.4737 | 0.6842 |
| bert-large-nli-stsb-mean-tokens | 0.4786 | 0.3684 | 0.4737 | 0.4737 |
| bert-base-nli-mean-tokens | 0.4422 | 0.2632 | 0.5263 | 0.6842 |
| roberta-large-nli-stsb-mean-tokens | 0.4368 | 0.2632 | 0.5263 | 0.6842 |
| roberta-base-nli-stsb-mean-tokens | 0.3804 | 0.2632 | 0.3158 | 0.5263 |
| distilbert-base-nli-stsb-mean-tokens | 0.3058 | 0.0526 | 0.4737 | 0.4737 |

Based on the provided table, the best model for (shorter) text similarity is “all-mpnet-base-v2” because it has the highest values across all key evaluation metrics:

- **Mean Reciprocal Rank (MRR): 0.7089 (highest among all models)**
- **Top-1 Accuracy: 0.5789 (highest)**
- **Top-3 Accuracy: 0.7895 (highest)**
- **Top-5 Accuracy: 0.8421 (highest)**

This indicates that **all-mpnet-base-v2** is the most accurate model **for shorter text similarity tasks**. MPNet-based models outperform others in short-form text similarity tasks due to their better contextual embeddings.

**Criteria 2 – Longer Text (Product Description)**

| Model Name | Mean Reciprocal Rank (MRR) | Top-1 Accuracy | Top-3 Accuracy | Top-5 Accuracy |
|---|---|---|---|---|
| bert-large-nli-stsb-mean-tokens | 0.6899 | 0.6182 | 0.7273 | 0.7636 |
| all-MiniLM-L6-v2 | 0.6503 | 0.5455 | 0.7273 | 0.7818 |
| all-MiniLM-L12-v2 | 0.6409 | 0.5455 | 0.7091 | 0.7636 |
| all-mpnet-base-v2 | 0.6294 | 0.5273 | 0.6727 | 0.7818 |
| paraphrase-multilingual-MiniLM-L12-v2 | 0.6236 | 0.5455 | 0.6727 | 0.7091 |
| roberta-large-nli-stsb-mean-tokens | 0.6107 | 0.5091 | 0.6909 | 0.6909 |
| roberta-base-nli-stsb-mean-tokens | 0.6106 | 0.5273 | 0.6364 | 0.7091 |
| bert-base-nli-mean-tokens | 0.5884 | 0.4727 | 0.6545 | 0.7091 |
| distilbert-base-nli-stsb-mean-tokens | 0.5713 | 0.4545 | 0.6545 | 0.7091 |

Based on the table, "bert-large-nli-stsb-mean-tokens" is the best model for ensuring correct product matches appear as the first result because:

1. **Highest Mean Reciprocal Rank (MRR)**:
   - MRR measures ranking quality, particularly how early the correct match appears.
   - The highest MRR (0.6899) suggests this model ranks correct matches higher than others.
2. **Highest Top-1 Accuracy**:
   - This indicates how often the correct match appears as the first result.
   - At **0.6182**, it outperforms all other models in getting the first result correct.

For longer text similarity tasks, BERT-based models (especially the larger versions) perform well due to their ability to capture contextual information effectively. Since **bert-large-nli-stsb-mean-tokens** leads in both key metrics, it is the best choice **for longer text similarity tasks**.

The above results further confirm that ***all-mpnet-base-v2*** and ***bert-large-nli-stsb-mean-tokens*** are the top-performing models for Product Name/Category and Product Description respectively among the evaluated SBERT models, as also validated by the STS Benchmark.

Once a product inquiry is submitted, the ML system will process it using two models: all-mpnet-base-v2 for analyzing the Product Name/Category and bert-large-nli-stsb-mean-tokens for evaluating the Product Description.

- If both models identify the same product, its category will be returned directly to the backend.
- If the models return different products, the system will select the one with the highest similarity score and return its category to the backend.

The retrieved product category will then be stored in Azure Tables and made available for the administrator's review and approval within the web portal.

## Model Selection for Image Similarity

I researched pretrained CNNs like ResNet50, VGG16/VGG19, InceptionV3, DenseNet, EfficientNet, and MobileNetV2, which are used for image classification. They are trained on large datasets like ImageNet and can be fine-tuned for tasks like object detection or segmentation. Their designs vary, with features like skip connections (ResNet), deeper layers (VGG), and efficient scaling (EfficientNet, MobileNetV2).

- **Residual Networks**: ResNet introduces residual connections, allowing gradients to flow more easily through deep networks – ***ResNet50***
- **Simple Deep Networks**: Uses a stack of 3x3 convolution layers followed by max-pooling layers, making the architecture easy to understand – ***VGG16 / VGG19***
- **Inception Networks**: A key feature is the "Inception module" that performs multiple convolutions with different filter sizes in parallel, followed by concatenation - ***InceptionV3***
- **Dense Networks**: Each layer receives input from all previous layers (dense connections), allowing the model to reuse features more efficiently - ***DenseNet***
- **Efficient Networks**: EfficientNet uses a compound scaling method that scales depth, width, and resolution uniformly to achieve better efficiency - ***EfficientNet***
- **Lightweight Networks for Mobile Devices**: MobileNets use depthwise separable convolutions to reduce the number of parameters and computation, making them efficient for mobile and embedded applications - ***MobileNetV2***

| Model | Advantages | Disadvantages | Best For | Example Use Case |
|---|---|---|---|---|
| **ResNet50** | • Strong performance with deep layers.<br>• Efficient feature extraction with pre-trained weights.<br>• Works well for transfer learning. | • More complex than shallow networks.<br>• May not capture small-scale features as well as more specialized models. | • General image similarity<br>• Transfer learning | • E-commerce product recommendation<br>• Content-based image retrieval |
| **VGG16 / VGG19** | • Simple and easy to understand.<br>• Strong transfer learning capability.<br>• Works well for large datasets when fine-tuned. | • High computational cost.<br>• Lack of advanced optimization techniques like residual connections. | • General image classification<br>• Transfer learning | • Flower identification<br>• Object classification |
| **InceptionV3** | • Efficient and scalable.<br>• Multi-scale feature extraction using Inception modules.<br>• Suitable for large datasets. | • More complex architecture.<br>• Requires more memory and computational resources. | • Large-scale datasets<br>• Multi-scale feature extraction | • Facial recognition<br>• Product search |
| **DenseNet** | • Efficient parameter usage.<br>• Improved gradient flow.<br>• Strong feature reuse. | • Memory-intensive due to dense connections.<br>• Slower training. | • Image classification<br>• Medical image analysis | • Tumor detection<br>• Image segmentation |
| **EfficientNet** | • Best trade-off between accuracy and efficiency.<br>• Scalable across models. | • Complex to implement.<br>• Might not always outperform simpler models in small datasets. | • General-purpose image recognition<br>• Large-scale image tasks | • High-accuracy image classification<br>• Image retrieval |
| **MobileNetV2** | • Lightweight and efficient.<br>• Low latency for mobile and edge devices. | • Lower accuracy compared to larger models.<br>• Limited to simpler tasks. | • Mobile applications<br>• Real-time inference | • Augmented reality<br>• Mobile product search |

## Why ResNet50?

- **Deep architecture with residual connections**: **ResNet50**’s deep structure and residual connections allow it to capture complex features and perform better at distinguishing subtle image differences, unlike **VGG16/VGG19**, which lacks residuals and struggles with deep networks.

- **Feature extraction efficiency**: **ResNet50** balances depth and computational cost effectively, whereas **InceptionV3** is more complex and requires more resources, and **EfficientNet** requires fine-tuning for similarity tasks.
- **Pretrained models**: **ResNet50**'s pretrained models, particularly on ImageNet, provide strong feature representations for image similarity tasks, unlike **DenseNet**, which is memory-intensive, and **MobileNetV2**, which sacrifices depth for efficiency.
- **Computational efficiency**: ResNet50 offers a good trade-off between performance and cost, whereas **MobileNetV2** is optimized for mobile devices and **EfficientNet** is efficient but requires extra optimization for similarity tasks.

## Feedback Collected from User Interviews

### Completed Features

1. **Ability to sort the submitted/pending/approved product inquiry tables –** ***Done***
    - **Explanation:** Users can now sort product inquiries by various columns such as Submitted Date, Approved Date, User Email and Product Name. This helps users quickly find relevant entries and improves the overall usability of the system.
2. **Ability to filter the submitted/pending/approved product inquiry tables –** ***Done***
    - **Explanation:** Filtering options have been added to help users narrow down inquiries based on specific criteria like Submitted Date, Approved Date, User Email, Product Name, Description, Recommended Category and Approved Category. This allows for more efficient navigation and better data management.

### Planned for Future Phases

3. **Ability to edit the submitted product inquiry by the user –** ***Will be done in the next phase***
    - **Explanation:** Users will soon have the capability to update previously submitted inquiries in case of mistakes or updates. This will reduce back-and-forth communication and improve data accuracy.
4. **Enable notifications via email when a product inquiry is submitted/approved –** ***Will be done in the next phase***

- **Explanation:** Email notifications will be sent to users when their product inquiry is successfully submitted or when it receives approval. This keeps users informed and reduces the need to manually check the portal for updates.

5. **Most product inquiries include a link (e.g., Amazon) instead of a description. We need to scrape the linked webpage to extract relevant product data for the ML model – *Will be done in the next phase***

    - **Explanation:** Since users often submit URLs instead of detailed product descriptions, a web scraping feature will be introduced to automatically extract essential product details (e.g., product name, product description and product image) from the linked pages. This ensures that the ML model receives consistent and structured input data, improving its performance and predictions.

6. **Integrating the product into the members portal – *Will be done in the final phase***

    - **Explanation:** The entire product inquiry functionality will be seamlessly integrated into the members' portal, providing a unified and user-friendly experience. This includes UI/UX improvements, single sign-on, and alignment with the broader platform's ecosystem.

## Evaluating the Product using Past Product Inquiries

Based on the testing with 50 past product inquiries that were relatively straightforward, it was observed that the implemented solution significantly improves the current manual workflow. Specifically, it **reduces the average response time by approximately 80%**, demonstrating a major efficiency gain.

However, for more complex or ambiguous inquiries, human intervention is still necessary to ensure accuracy and context-specific understanding. These cases often involve incomplete data, non-standard product descriptions, or subjective decision-making that the current model cannot yet handle effectively.

As it continues to gather more data and fine-tune the machine learning model, the goal is to gradually minimize the need for manual intervention, even in non-straightforward scenarios. Over time, this will lead to a more autonomous and scalable system capable of handling a wider range of product inquiries with high confidence.

# Reflections / Discussions

## Insights Gathered

- Gained a detailed understanding of the product inquiry process and how it connects various components across the business workflow.
- Building the web portal before implementing the ML models provided a clear, end-to-end view of the system, which simplified model integration and helped visualize the big picture.

## Challenges Faced and Lessons Learned

- **Challenge:** CNN model - ResNet50 was slow, taking over 5 minutes to run.
    - **Lesson:** Precomputing and storing embeddings significantly reduce runtime (from 5 minutes to 5 seconds), highlighting the importance of performance optimization in production systems.
- **Challenge:** Limited experience using Python in a full-stack development context.
    - **Lesson:** Leveraging Python Flask resources and integrating tools like OpenAI allowed for rapid upskilling and effective backend development.
- **Challenge:** Understanding and applying complex NLP and CNN models.
    - **Lesson:** Persistence and self-learning through various online articles and tutorials made it possible to grasp and apply these advanced concepts.
- **Challenge:** Insufficient labeled data for training ML models.
    - **Lesson:** Introducing an admin review process in the web portal not only improves data quality but also creates a feedback loop to accumulate valuable training data over time.

## Most Satisfying Parts of the Project

- Built a fully functional, end-to-end, AI-integrated full-stack product for the first time.
- Shifted from doing academic or ad hoc AI projects to creating a real-world solution that directly benefits the employer.

# Work Logs

| Phase | Date | Number of Hours | Description of Work Done |
|---|---|---|---|
| Final Report | 07/04/2025 | 1 | Modified the ml_model script to change how it shows the matched category in the terminal |
| Final Report | 08/04/2025 | 2 | Modified the html scripts to search/filter the submitted/pending/approved products tables by selected fields |
| Final Report | 09/04/2025 | 2 | Student Research Days 2025 preparation |
| Final Report | 10/04/2025 | 4 | Student Research Days 2025 - Presenting the research under research poster category |
| Final Report | 11/04/2025 | 1 | Prepared the complete list of References |
| Final Report | 12/04/2025 | 3 | Drafted and finalized the final report & presentation |
| Final Report | 13/04/2025 | 3 | Drafted and finalized the final report & presentation |

Above are the logs detailing the work completed after submitting Progress Report 5. For a complete view of all work logs, please refer to this Google Sheet.

# Conclusion

This applied research project was both challenging and rewarding. Despite facing several knowledge gaps and time constraints, I was able to quickly adapt and learn on the fly to successfully build the product as initially proposed. The process pushed me to step outside my comfort zone, particularly in areas like NLP and CNN, which are at the forefront of today's AI advancements. These technologies played a significant role in the project, and gaining hands-on experience with them was invaluable.

The journey also taught me the importance of persistence, continuous learning, and adapting quickly to unforeseen obstacles. Although it was my first time building a full-stack AI product, the result was a functional and impactful solution that addressed real business needs. This project has not only deepened my technical skills but also reinforced my passion for AI and its potential to transform industries. Ultimately, this experience has provided me with a deeper understanding of how to deliver high-impact AI products under tight deadlines and has set a strong foundation for future endeavors in AI development.

# Acknowledgements

I would like to express my gratitude to Product Care Association (PCA) for providing the data and supporting the development of this project. I also extend my appreciation to my manager, *Luther Trammell,* as well as Zeus Baral, Mario Anda, and Roxanne Yip for their guidance and valuable insights. Special thanks to my instructor, *Padmapriya Arasanipalai Kandhadai*, for her continuous feedback and support.

# References

## AI Tools

- ChatGPT: OpenAI (paid version)

  Used for these purposes:

  - Used for research on Natural Language Processing (NLP) and Convolutional Neural Network (CNN) models
  - Provided assistance with coding and debugging
  - Supported writing, editing, and refining the report

## General References


- PCA Official Website: https://www.productcare.org
- How are lights and light bulbs recycled: https://www.youtube.com/watch?v=KHmwDosH-FQ
- PCA Light Recycling Product Guide (Product Catalogue)
- Python Tutorial W3Schools - https://www.w3schools.com/python/
- Flask Documentation (3.1.x): https://flask.palletsprojects.com/en/stable/
- Bootstrap-Flask - Bootstrap-Flask 2.2.x documentation - https://bootstrap-flask.readthedocs.io/en/stable/
- Bootstrap 5.3.0 - https://blog.getbootstrap.com/2023/05/30/bootstrap-5-3-0/
- Bootstrap CDN files (bootstrap.min.css) - https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/
- NLP & BERT-based Text Matching: https://huggingface.co/transformers/
- Computer Vision with ResNet: https://pytorch.org/hub/pytorch_vision_resnet/
- FAISS for Similarity Search: https://faiss.ai/
- Pretrained Models - Sentence Transformers documentation: https://www.sbert.net/docs/sentence_transformer/pretrained_models.html
- Large Language Models: SBERT – Sentence-BERT: https://towardsdatascience.com/sbert-deb3d4aef8a4/
- Hugging Face Models: https://huggingface.co/models
- What is Sentence Similarity: https://huggingface.co/tasks/sentence-similarity
- Hugging Face Sentence Similarity Models: https://huggingface.co/models?pipeline_tag=sentence-similarity&sort=trending

- Natural Language Processing: NLP With Transformers in Python Udemy Course: https://www.udemy.com/course/nlp-with-transformers/
- Vector Similarity Search and FAISS YouTube Course: https://www.youtube.com/playlist?list=PLIUOU7oqGTLhlWpTz4NnuT3FekouIVlqc
- Understanding Image Similarity with Machine Learning: https://medium.com/@aleksej.gudkov/understanding-image-similarity-with-machine-learning-c8680c24dd56
- High Similarity Image Recognition and Classification Algorithm Based on Convolutional Neural Network: https://onlinelibrary.wiley.com/doi/10.1155/2022/2836486
- Deep Learning Vs Traditional Computer Vision Techniques: Which Should You Choose?: https://medium.com/discover-computer-vision/deep-learning-vs-traditional-techniques-a-comparison-a590d66b63bd
- Navigating the Intersection of Traditional Computer Vision and Deep Learning: A Guide to Building Production Systems: https://www.strong.io/blog/navigating-the-intersection-of-traditional-computer-vision-and-deep-learning-a-guide-to-building-production-systems
- An Introduction to Convolutional Neural Networks (CNNs): https://www.datacamp.com/tutorial/introduction-to-convolutional-neural-networks-cnns
- Convolutional Neural Networks (CNN) in Deep Learning: https://www.analyticsvidhya.com/blog/2021/05/convolutional-neural-networks-cnn/
- Exploring ResNet50: An In-Depth Look at the Model Architecture and Code Implementation: https://medium.com/@nitishkundu1993/exploring-resnet50-an-in-depth-look-at-the-model-architecture-and-code-implementation-d8d8fa67e46f
- Hugging Face - SentenceTransformers Documentation: https://www.sbert.net/
- TensorFlow – Performance Guide for Serving Models: https://www.tensorflow.org/tfx/serving/performance
- MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications: https://arxiv.org/abs/1704.04861
- Facebook AI Research – FAISS GitHub: https://github.com/facebookresearch/faiss
- Celery Official Documentation: https://docs.celeryq.dev/en/stable/getting-started/introduction.html
- Python's *concurrent.futures* Documentation: https://docs.python.org/3/library/concurrent.futures.html

# Appendix A: Installation Guide

## Project Overview

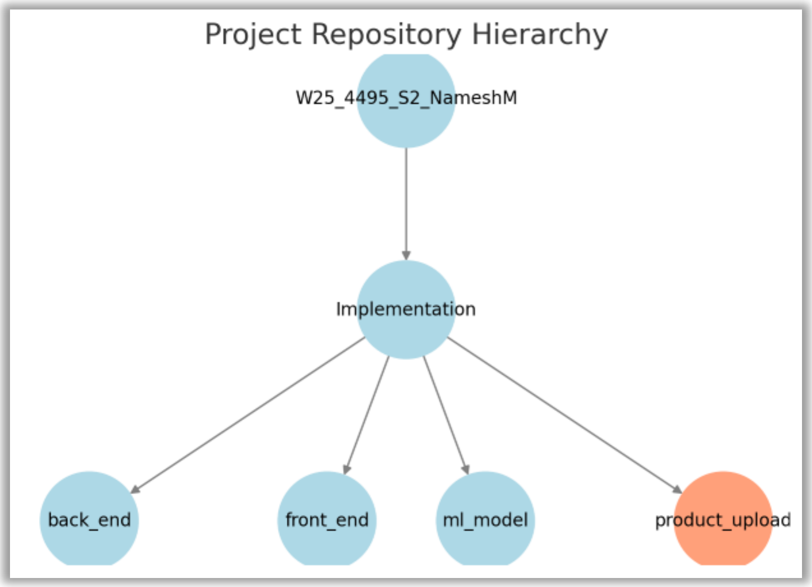
This repository (**W25_4495_S2_NameshM**) is a mono-repo containing the code for a Python-based application that automates product inquiry processing using machine learning. The project consists of a **Flask**-based frontend and backend, integrated with **Azure Blob Storage** for storing images and **Azure Table Storage** for text-based data.

## Technology Stack

- **Frontend**: Flask + Bootstrap (UI)
- **Backend**: Flask, Azure Blob Storage (image storage), Azure Table Storage (text data storage)
- **ML Model**: Python-based ML model for product similarity matching
- **Programming Language**: Python

## Project Structure

- *back_end/* - Contains the backend application
- *front_end/* - Contains the frontend application
- *ml_model/* - Contains the machine learning model
- *product_upload/* - Handles product guide data uploads - **This is a one-time task needs to be done at the beginning of the project**



## Installation Guide

### Prerequisites

Ensure you have the following installed:

- **Python (>=3.9)** - Install from [python.org]
- **pip** - Installed with Python
- **Git** - Install from [git-scm.com]

### Clone the Repository

```
git clone https://github.com/namesh89/W25_4495_S2_NameshM.git
```

### Setup Virtual Environment & Install Dependencies for Each Module

```
cd W25_4495_S2_NameshM/Implementation/back_end
python -m venv venv
# Activate virtual environment
# Windows
venv\Scripts\activate
# Mac/Linux
source venv/bin/activate
pip install -r requirements.txt

cd ../front_end
python -m venv venv
# Activate virtual environment
# Windows
venv\Scripts\activate
# Mac/Linux
source venv/bin/activate
pip install -r requirements.txt

cd ../ml_model
python -m venv venv
# Activate virtual environment
```

```
# Windows
venv\Scripts\activate
# Mac/Linux
source venv/bin/activate
pip install -r requirements.txt
```

## Configuration

**Set Up Environment Variables for Each Module**

**The actual values of these variables are NOT given here due to security concerns**

Create a *.env* file inside the *back_end* directory and configure the following:

```
AZURE_STORAGE_ACCOUNT_NAME=azure_storage_account_name
AZURE_STORAGE_ACCOUNT_KEY=azure_storage_account_key
AZURE_STORAGE_CONNECTION_STRING=azure_storage_connection_string
JWT_SECRET_KEY=jwt_secret_key
ML_MODEL_URL=ml_model_url
```

Create a *.env* file inside the *front_end* directory and configure the following:

```
SECRET_KEY=secret_key
BACKEND_URL=backend_url
AZURE_STORAGE_ACCOUNT=azure_storage_account
AZURE_STORAGE_KEY=azure_storage_key
AZURE_CONTAINER_NAME=azure_container_name
AZURE_UPLOAD_FOLDER=azure_upload_folder
```

Create a *.env* file inside the *ml_model* directory and configure the following:

```
SECRET_KEY=secret_key
BACKEND_URL=backend_url
APIRS_SERVICE_API_URL=apirs_service_api_url
AZURE_STORAGE_ACCOUNT_NAME=azure_storage_account_name
AZURE_STORAGE_ACCOUNT_KEY=azure_storage_account_key
AZURE_STORAGE_CONNECTION_STRING=azure_storage_connection_string
```

## Creating user accounts

**Creating admin account**

*cd back_end*

*python add_user.py*

Enter *admin@test.com* as email when prompted and a suitable password.

**Creating member accounts**

*python add_user.py*

Enter member's email (e.g. *member1@test.com*) as email when prompted and a suitable password.

## Running the Applications

**Start the Backend in a new terminal**

*cd back_end*

*python app.py*

The backend should now be running on *http://127.0.0.1:5000*

**Start the Frontend in a new terminal**

*cd front_end*

*python app.py*

The frontend should now be running on *http://127.0.0.1:5001*

**Start the ML Model API in a new terminal**

*cd ml_model*

*python app.py*

The ML Model API should now be running on *http://127.0.0.1:5002*

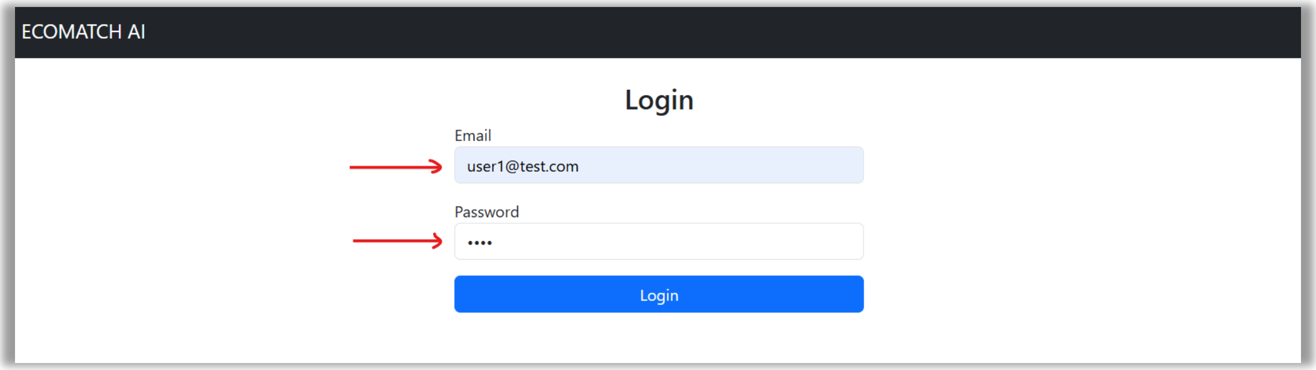
## Running a Demo

1. **Upload a Product Inquiry**
    - Use the frontend UI (*http://127.0.0.1:5001*) to login as a member and submit a product inquiry (name, description, and image). You can also view the submitted product inquiries and approved product inquiries.
2. **Processing the Inquiry**
    - The backend API (*http://127.0.0.1:5000*) processes the inquiry and store the data in Azure Table Storage (text) and Azure Blob Storage (image) then submits those data to ML model.
    - The ML model (*http://127.0.0.1:5002*) performs text and image similarity matching to find the most-matched product from existing data and then passes its product category to update the same Azure Table Storage.
3. **Approve the Product Category**
    - Use the frontend UI (*http://127.0.0.1:5001*) to login as the admin and view pending product inquiries submitted by members. You can then review and approve the product category passed by the ML model. Once you approve, you can also view the approved product inquiries.

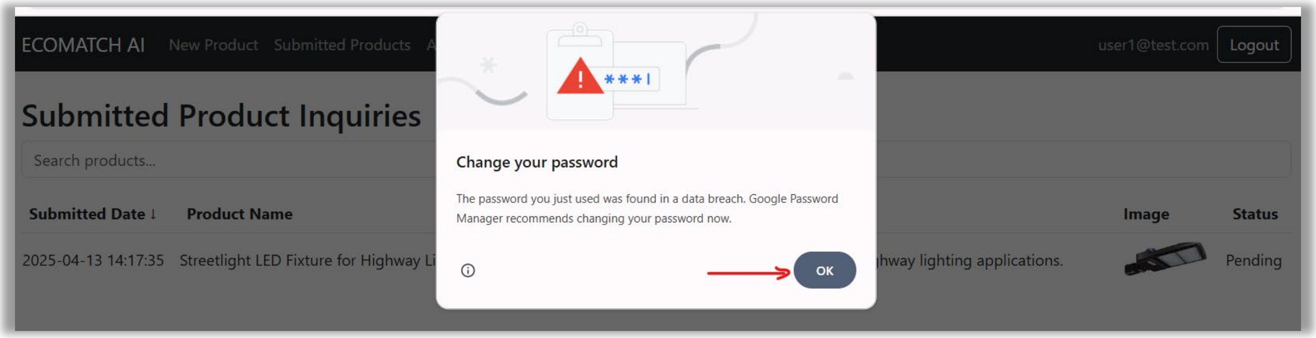
# Appendix B: User Guide

## As a Member

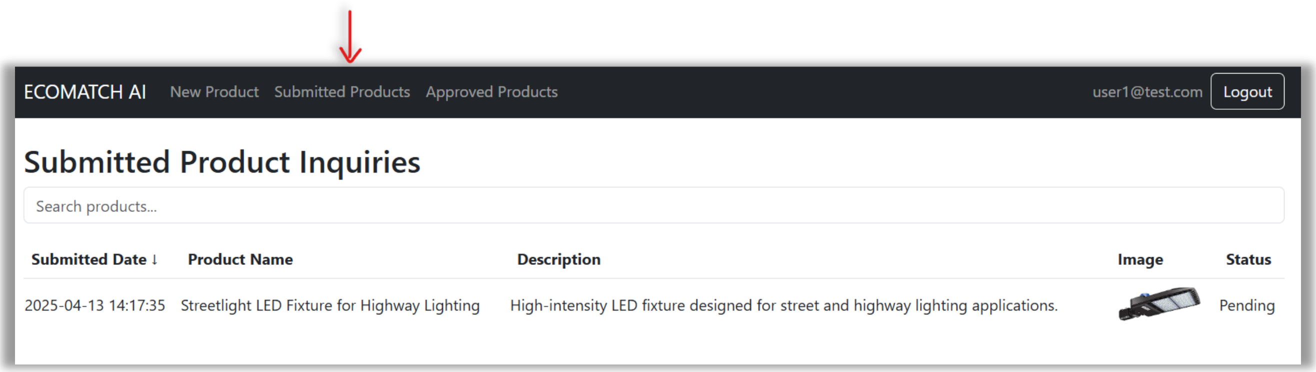
1. Enter your member credentials on the Login page and click Login.



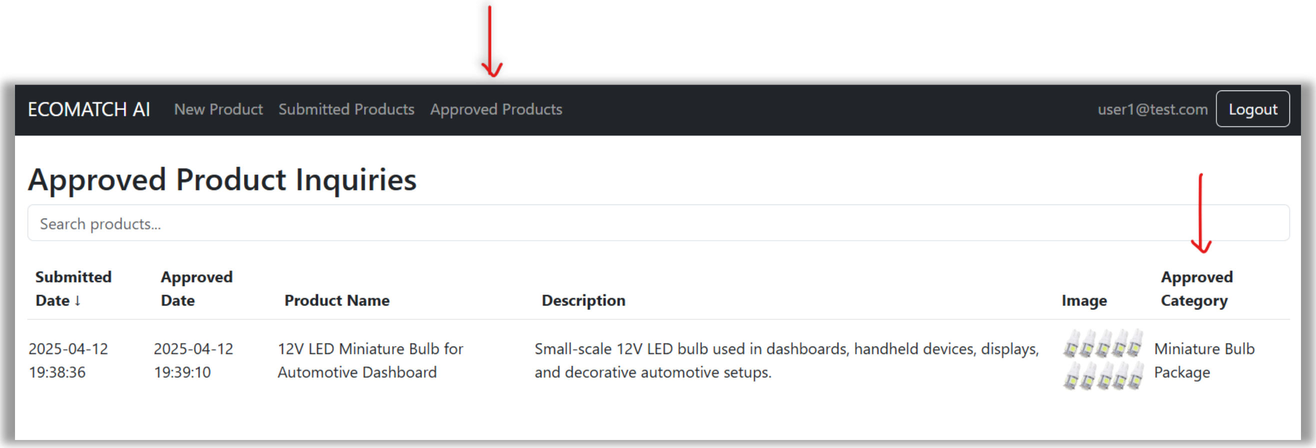

2. Click OK.



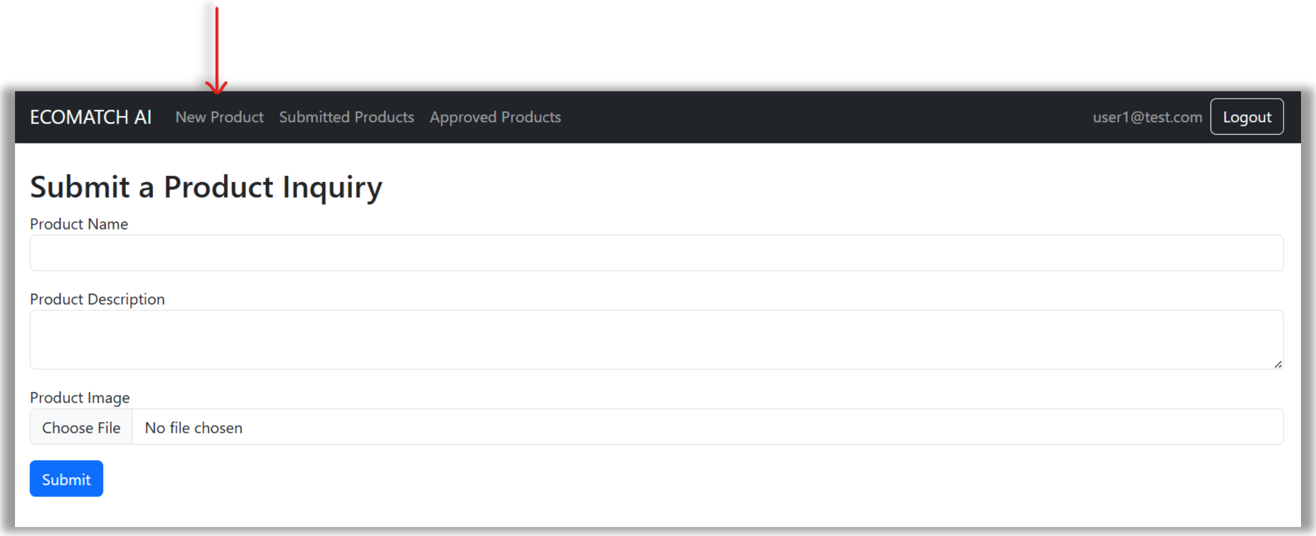

3. Now you can see the past submitted product inquiries, but not yet approved.



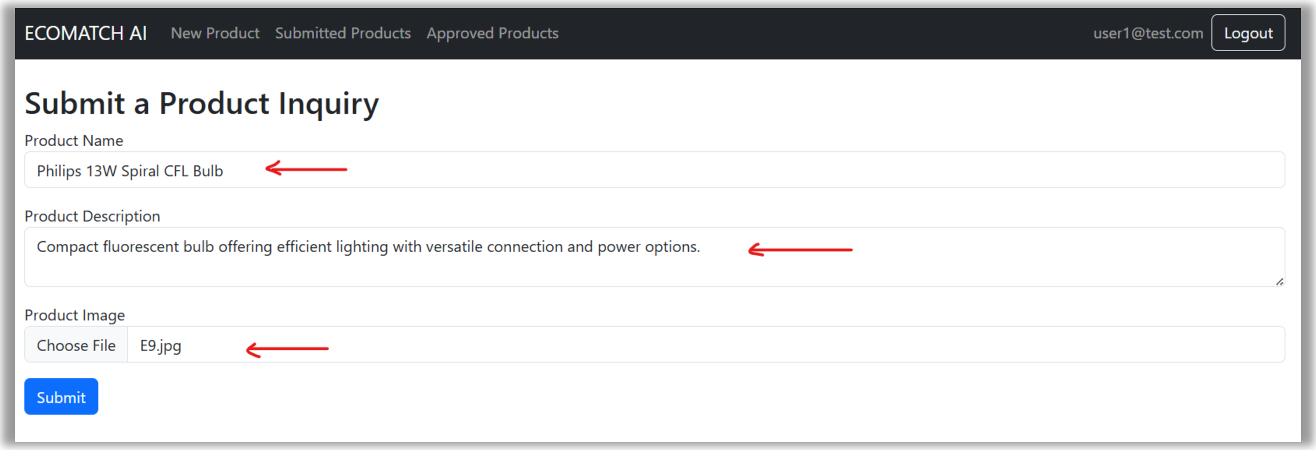
4. Click Approved Products tab to see the approved product inquiries. Check the Approved Category to get the correct product category that the product belongs to.



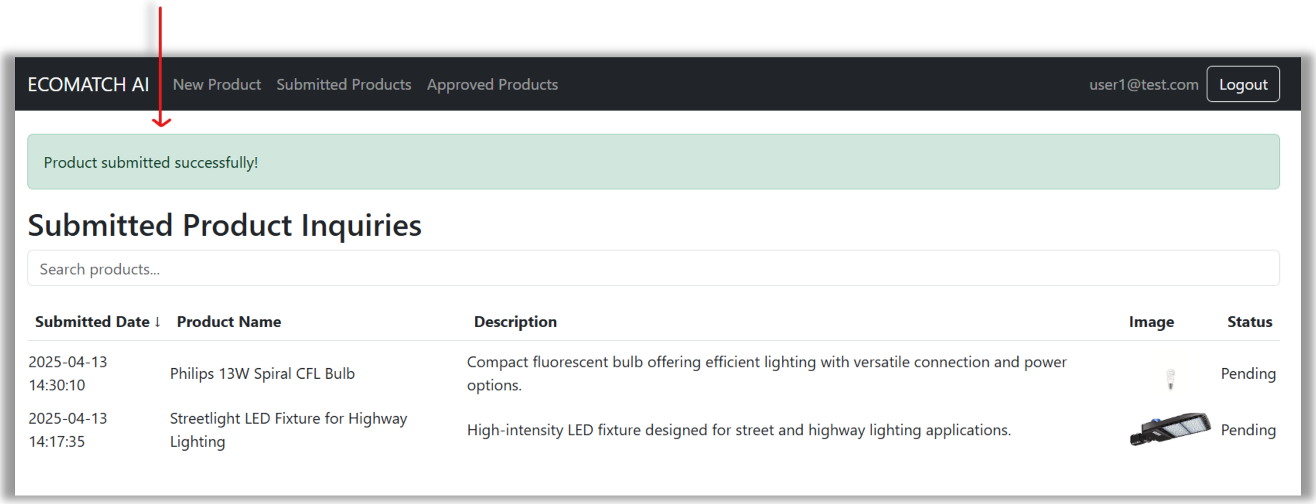

5. To submit a new product inquiry, click the New Product tab.



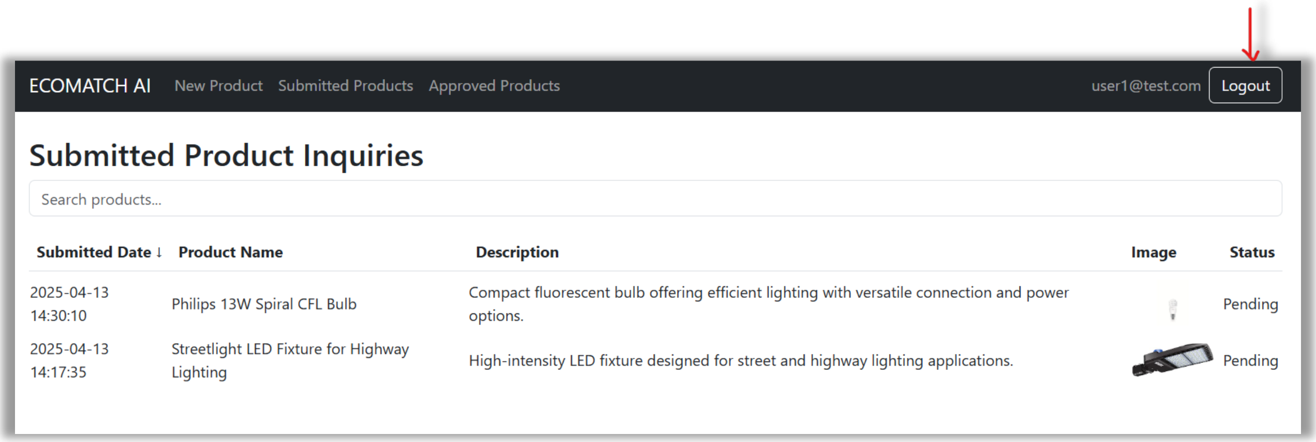

6. Enter the Product Name, Description in the relevant fields and upload a picture of the product. Then click Submit.



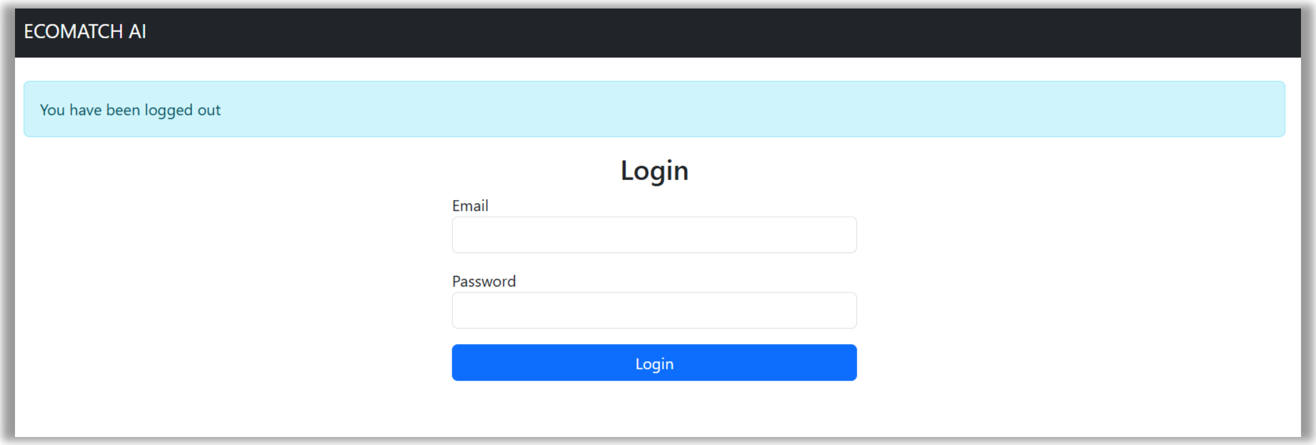
7. Once submitted, you will see a green notification "Product submitted successfully".




8. To log out from the portal, click Logout button.

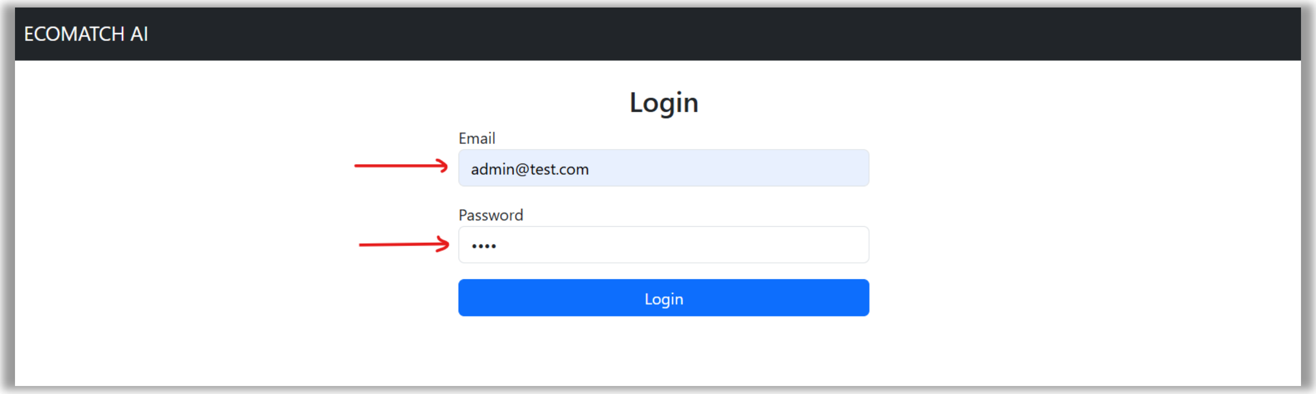



9. Once logged out, it will redirect you to the Login page.

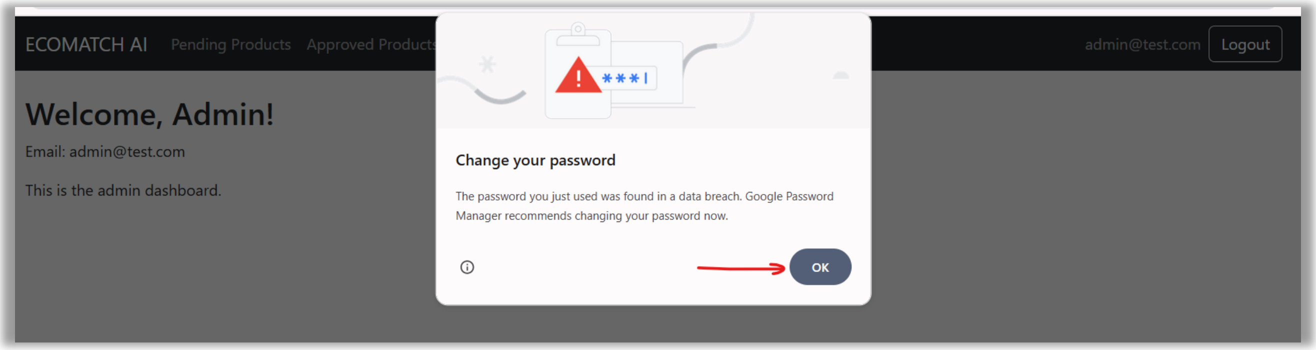


## As the PCA Administrator

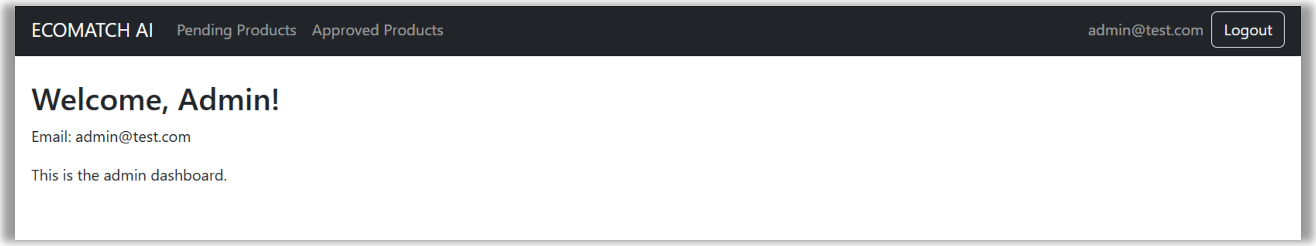
1. Enter your admin credentials on the Login page and click Login.



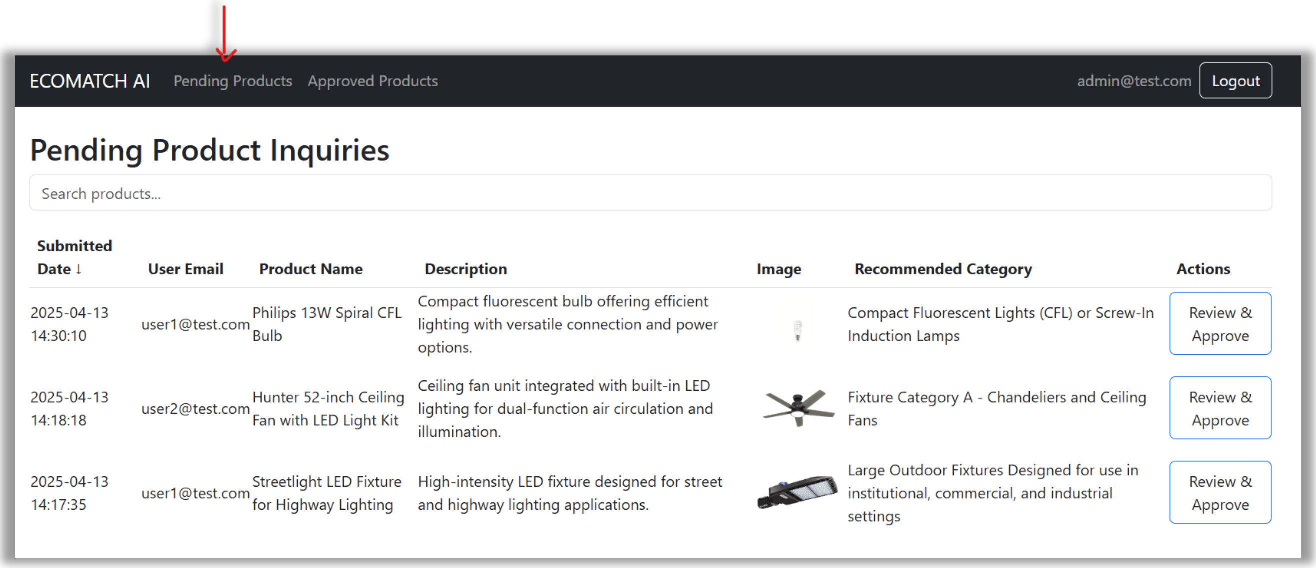

2. Click OK.




3. Now you can see the Welcome page of the admin portal.



4. Click Pending Products tab to see the submitted product inquiries by different members, but not yet approved.

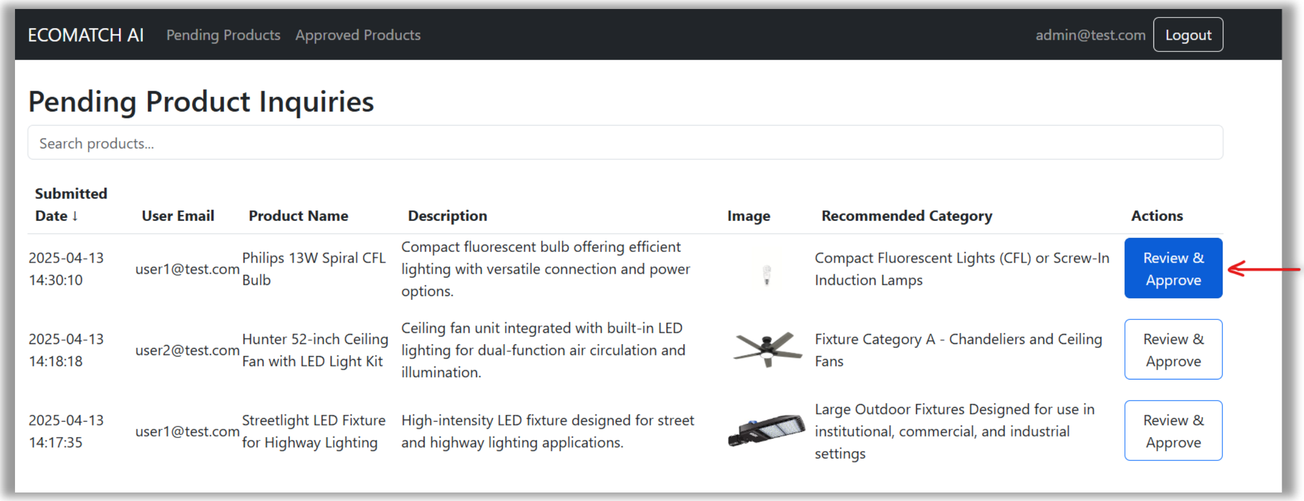

ECOMATCH AI Pending Products Approved Products admin@test.com Logout

Pending Product Inquiries

Search products...

| Submitted Date ↓ | User Email | Product Name | Description | Image | Recommended Category | Actions |
|---|---|---|---|---|---|---|
| 2025-04-13 14:30:10 | user1@test.com | Philips 13W Spiral CFL Bulb | Compact fluorescent bulb offering efficient lighting with versatile connection and power options. | | Compact Fluorescent Lights (CFL) or Screw-In Induction Lamps | Review & Approve |
| 2025-04-13 14:18:18 | user2@test.com | Hunter 52-inch Ceiling Fan with LED Light Kit | Ceiling fan unit integrated with built-in LED lighting for dual-function air circulation and illumination. | | Fixture Category A - Chandeliers and Ceiling Fans | Review & Approve |
| 2025-04-13 14:17:35 | user1@test.com | Streetlight LED Fixture for Highway Lighting | High-intensity LED fixture designed for street and highway lighting applications. | | Large Outdoor Fixtures Designed for use in institutional, commercial, and industrial settings | Review & Approve |

5. Click Review & Approve button to review and approve the product category recommended by the ML model. Once clicked, it will guide you to a new page.

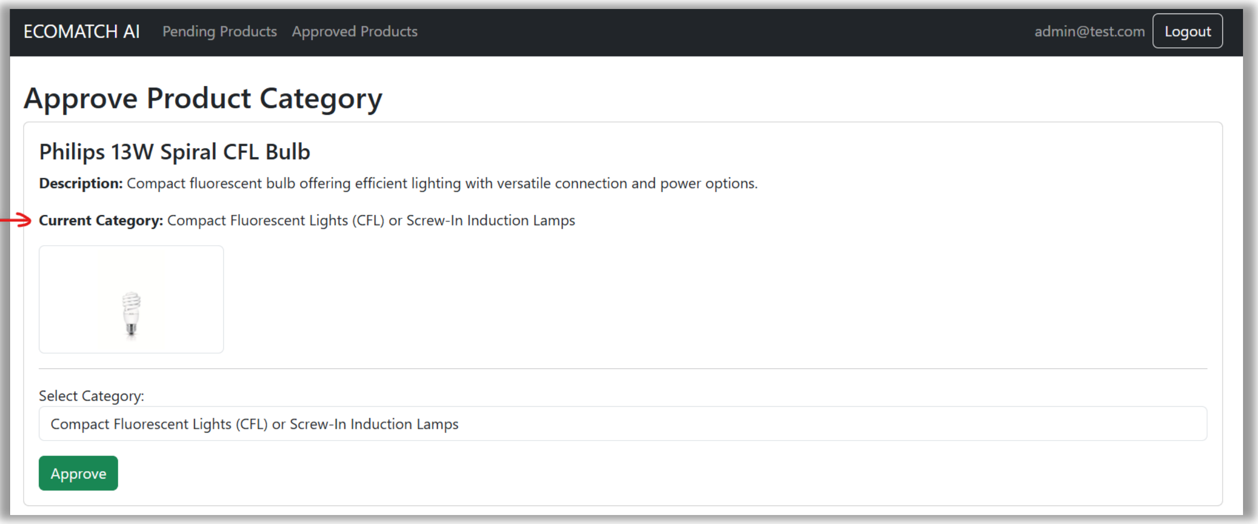

ECOMATCH AI Pending Products Approved Products admin@test.com Logout

Pending Product Inquiries

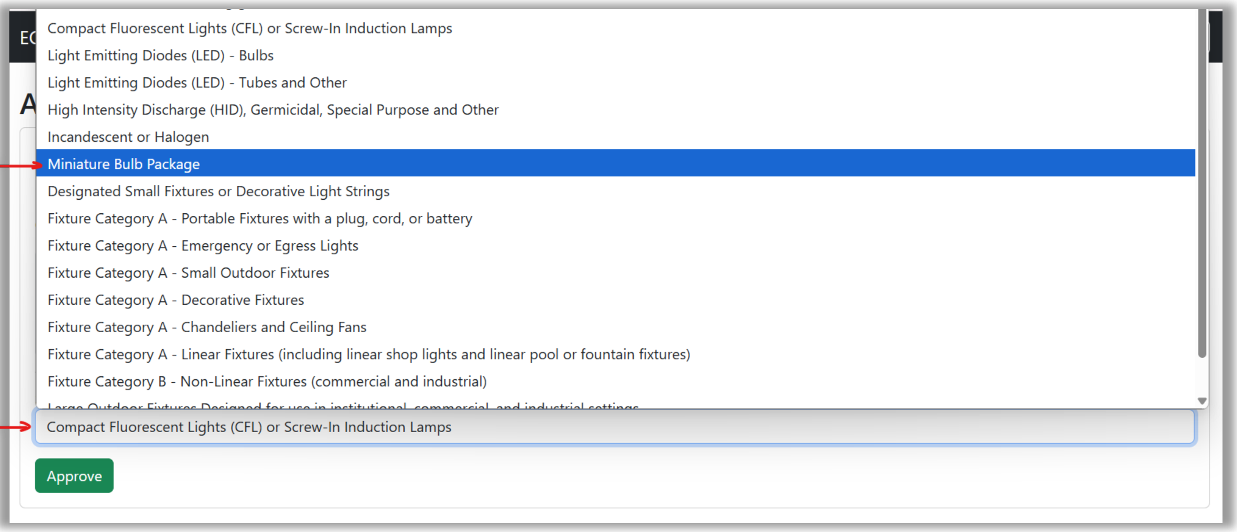
Search products...

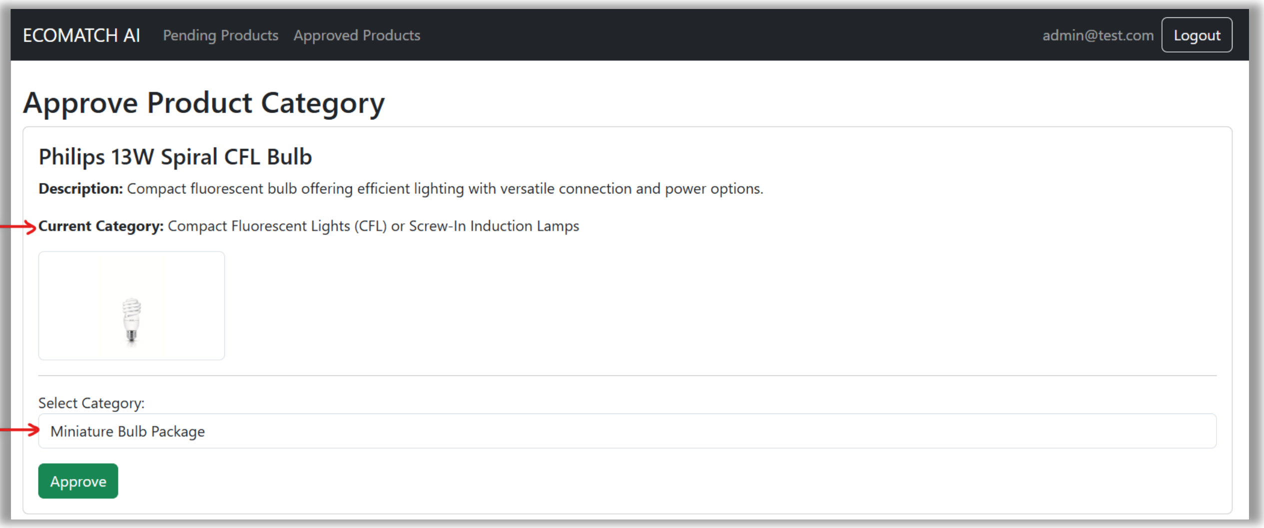
| Submitted Date ↓ | User Email | Product Name | Description | Image | Recommended Category | Actions |
|---|---|---|---|---|---|---|
| 2025-04-13 14:30:10 | user1@test.com | Philips 13W Spiral CFL Bulb | Compact fluorescent bulb offering efficient lighting with versatile connection and power options. | | Compact Fluorescent Lights (CFL) or Screw-In Induction Lamps | Review & Approve |
| 2025-04-13 14:18:18 | user2@test.com | Hunter 52-inch Ceiling Fan with LED Light Kit | Ceiling fan unit integrated with built-in LED lighting for dual-function air circulation and illumination. | | Fixture Category A - Chandeliers and Ceiling Fans | Review & Approve |
| 2025-04-13 14:17:35 | user1@test.com | Streetlight LED Fixture for Highway Lighting | High-intensity LED fixture designed for street and highway lighting applications. | | Large Outdoor Fixtures Designed for use in institutional, commercial, and industrial settings | Review & Approve |

6. If the Current Category (recommended by ML model) is correct, just click Approve.

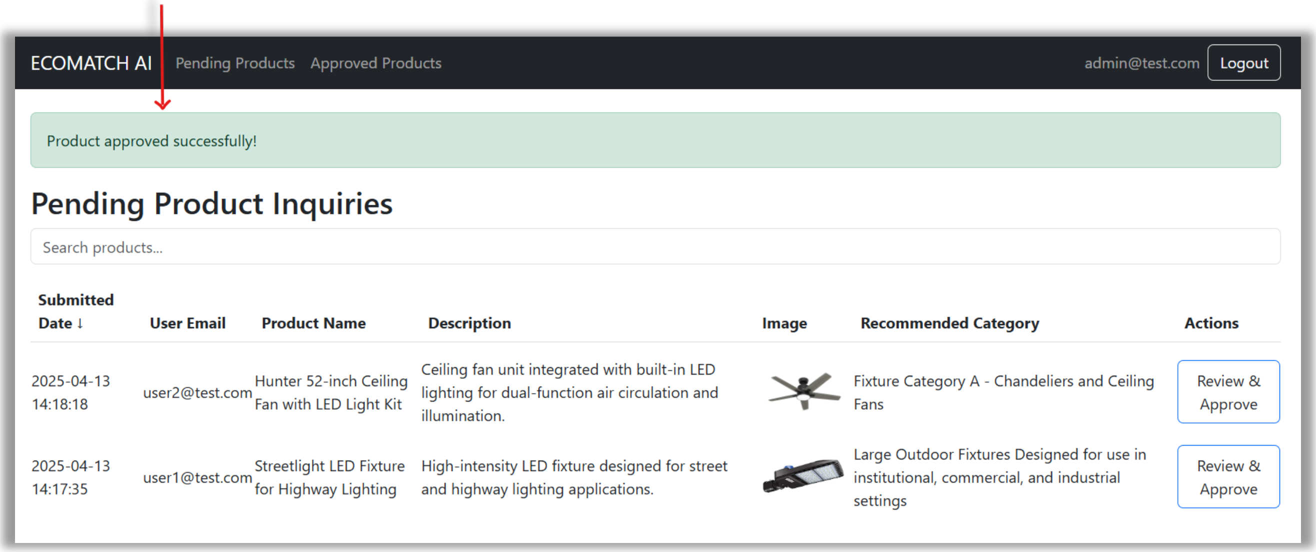



7. If the Current Category (recommended by ML model) is NOT correct, select the correct category from the drop-down menu, then click Approve.



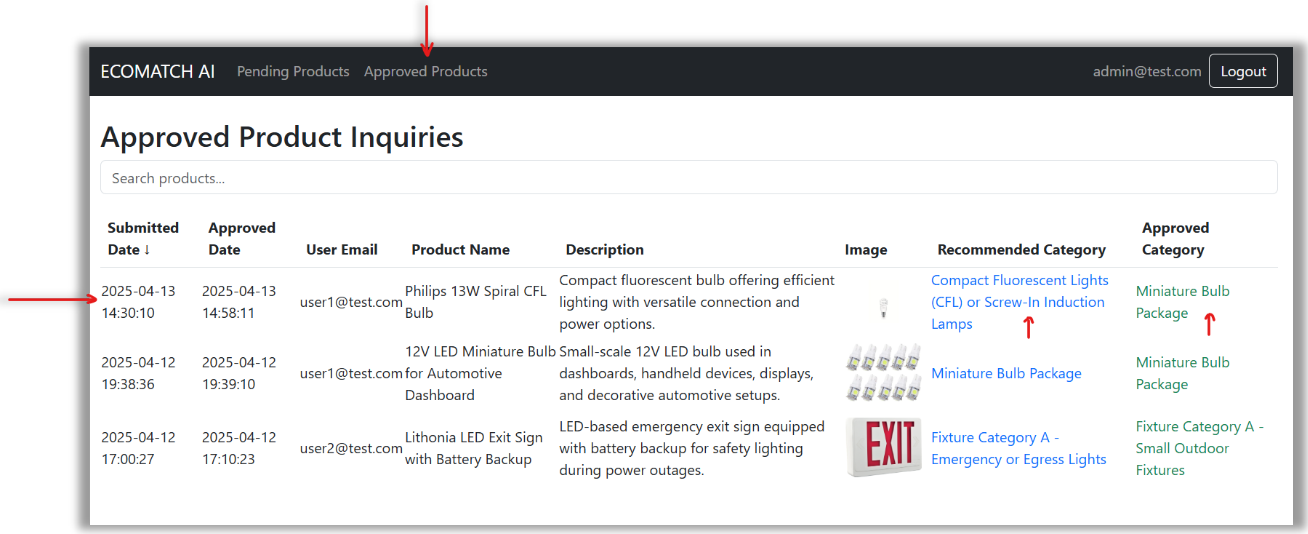



8. Once approved, you will see a green notification “Product approved successfully”.



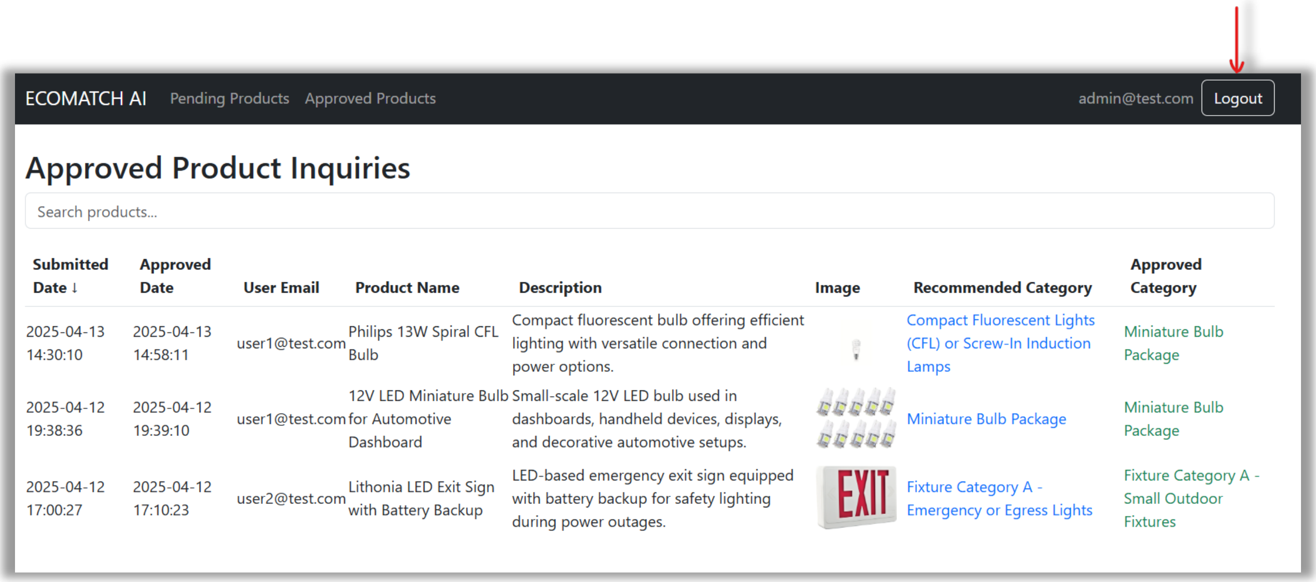

| Submitted Date ↓ | User Email | Product Name | Description | Image | Recommended Category | Actions |
|---|---|---|---|---|---|---|
| 2025-04-13 14:18:18 | user2@test.com | Hunter 52-inch Ceiling Fan with LED Light Kit | Ceiling fan unit integrated with built-in LED lighting for dual-function air circulation and illumination. | | Fixture Category A - Chandeliers and Ceiling Fans | Review & Approve |
| 2025-04-13 14:17:35 | user1@test.com | Streetlight LED Fixture for Highway Lighting | High-intensity LED fixture designed for street and highway lighting applications. | | Large Outdoor Fixtures Designed for use in institutional, commercial, and industrial settings | Review & Approve |

9. Click Approved Products tab to see the approved product inquiries. Check how Recommended Category and Approved Category are different as per the step 7.




| Submitted Date ↓ | Approved Date | User Email | Product Name | Description | Image | Recommended Category | Approved Category |
|---|---|---|---|---|---|---|---|
| 2025-04-13 14:30:10 | 2025-04-13 14:58:11 | user1@test.com | Philips 13W Spiral CFL Bulb | Compact fluorescent bulb offering efficient lighting with versatile connection and power options. | | Compact Fluorescent Lights (CFL) or Screw-In Induction Lamps | Miniature Bulb Package |
| 2025-04-12 19:38:36 | 2025-04-12 19:39:10 | user1@test.com | 12V LED Miniature Bulb for Automotive Dashboard | Small-scale 12V LED bulb used in dashboards, handheld devices, displays, and decorative automotive setups. | | Miniature Bulb Package | Miniature Bulb Package |
| 2025-04-12 17:00:27 | 2025-04-12 17:10:23 | user2@test.com | Lithonia LED Exit Sign with Battery Backup | LED-based emergency exit sign equipped with battery backup for safety lighting during power outages. | | Fixture Category A - Emergency or Egress Lights | Fixture Category A - Small Outdoor Fixtures |

10. To log out from the portal, click Logout button.




11. Once logged out, it will redirect you to the Login page.